



AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE

Wydział Inżynierii Mechanicznej i Robotyki

KATEDRA ROBOTYKI I MECHATRONIKI

Projekt dyplomowy

***Projekt i wykonanie systemu lokalizacji pojazdu
Design and Implementation of a Vehicle Localization
System***

Autor:	<i>Miłosz Stec</i>
Kierunek studiów:	Inżynieria Mechatroniczna
Opiekun pracy:	<i>dr inż. Grzegorz Góra</i>

Kraków, 2025/2026

Spis treści

1.	Wstęp	4
2.	Cel i zakres pracy.....	6
2.1.	Szczegółowe cele pracy	6
2.2.	Zakres pracy.....	6
3.	Podstawy teoretyczne i przegląd rozwiązań	8
3.1.	Przegląd technologiczny	8
3.1.1.	Magistrala CAN	8
3.1.2.	Protokół MQTT	10
3.1.3.	Systemy GNSS i standard NMEA 0183	11
3.1.4.	Magistrala 1-Wire	13
3.1.5.	FreeRTOS	14
3.2.	Analiza wybranych rozwiązań rynkowych.....	14
3.2.1.	Lokalizator GPS BearPoint C18	15
3.2.2.	Lokalizator GPS OBD MK08	16
3.2.3.	Lokalizator GPS MK6B 4G.....	17
3.2.4.	Kategoryzacja dostępnych rozwiązań.....	18
3.2.5.	Wnioski z przeglądu i uzasadnienie wyboru technologii	19
4.	Projekt i implementacja urządzenia.....	20
4.1.	Założenia projektowe.....	20
4.1.1.	Założenia funkcjonalne	20
4.1.2.	Założenia sprzętowe	21
4.1.3.	Założenia programowe	21
4.2.	Architektura sprzętowa	22
4.2.1.	Jednostka centralna	22
4.2.2.	Układy peryferyjne	23
4.2.3.	Układ zasilania.....	24
4.3.	Architektura oprogramowania	25
4.3.1.	Organizacja kodu źródłowego i struktura plików	25
4.3.2.	Podział na zadania	26
4.3.3.	Użyte biblioteki.....	29
4.3.4.	Format danych i technika X-Macro	30
4.3.5.	Zaawansowana obsługa pamięci.....	31
4.3.6.	Mechanizmy synchronizacji i komunikacji międzyprocesowej	32
4.3.7.	Zarządzanie danymi i tryb Offline.....	32
4.3.8.	Synchronizacja czasu	33

4.4.	Integracja z platformą ThingsBoard	33
4.4.1.	Przetwarzanie danych i Silnik Reguł	33
4.4.2.	Wizualizacja danych i monitoring	34
4.4.3.	Dwukierunkowa komunikacja i zdalna konfiguracja	35
4.5.	Budowa prototypu.....	36
4.6.	Testy.....	40
4.6.1.	Weryfikacja elektryczna i uruchomieniowa	40
4.6.2.	Testy funkcjonalne oprogramowania.....	42
4.6.3.	Testy w warunkach eksploatacyjnych	45
4.5.4.	Ocena jakości danych GNSS	46
4.6.4.	Wizualizacja i telemetria	46
5.	Podsumowanie i wnioski	48
5.1.	Napotkane problemy i ograniczenia realizacyjne	48
5.2.	Wnioski z realizacji	50
5.3.	Kierunki dalszego rozwoju	50
6.	Wykaz Publikacji.....	52

1. Wstęp

Dynamiczny rozwój technologii Internetu Rzeczy (ang. IoT – Internet of Things) oraz postępująca miniaturyzacja układów elektronicznych redefiniują współczesne podejście do systemów monitorujących. Pod pojęciem Internetu Rzeczy rozumie się koncepcję, w której przedmioty fizyczne (rzeczy) są wyposażone w czujniki, oprogramowanie i inne technologie w celu łączenia się i wymiany danych z innymi urządzeniami i systemami za pośrednictwem Internetu [1]. Możliwość tworzenia zaawansowanych, a jednocześnie kompaktowych i energooszczędnych urządzeń znajduje szerokie zastosowanie w wielu gałęziach przemysłu, ze szczególnym uwzględnieniem logistyki i transportu.

Współczesne systemy lokalizacyjne ewoluowały z prostych rejestratorów pozycji do roli kompleksowych narzędzi telemetrycznych. Telemetria, będąca kluczowym elementem omawianego zagadnienia, to dziedzina techniki zajmująca się automatycznym pozyskiwaniem danych pomiarowych z trudno dostępnych miejsc oraz ich przesyłaniem drogą radiową lub przewodową do zewnętrznych systemów rejestrujących [2]. Pozwalają one nie tylko na śledzenie trasy pojazdu, ale także na akwizycję i analizę danych diagnostycznych oraz parametrów środowiskowych, co znacząco zwiększa ich wartość użytkową w zarządzaniu flotą.

Niniejsza praca inżynierska dotyczy projektu i realizacji autorskiego systemu lokalizacji pojazdu, opartego na platformie sprzętowej ESP32. Opracowany układ, działający w architekturze IoT, ma za zadanie rejestrować dane o położeniu geograficznym i zapisywać je lokalnie na karcie pamięci SD (Secure Digital), a po uzyskaniu dostępu do zaufanej sieci bezprzewodowej – synchronizować je z serwerem zewnętrznym. Projekt zakłada również opcjonalną integrację z magistralą pojazdu (CAN – Controller Area Network / OBDII – On Board Diagnostics II) w celu rejestracji podstawowych parametrów eksploatacyjnych oraz możliwość rozbudowy o czujniki środowiskowe.

Główną motywacją do podjęcia tematu była chęć pogłębienia wiedzy w zakresie systemów wbudowanych czasu rzeczywistego oraz praktycznego zastosowania protokołów komunikacyjnych w systemach rozproszonych. Istotnym czynnikiem decyzyjnym było również zidentyfikowane zapotrzebowanie rynkowe w ramach

rodzinnej firmy zajmującej się wynajmem pojazdów. Istniała tam potrzeba monitorowania eksploatacji floty, jednak dostępne rozwiązania komercyjne okazywały się nieekonomiczne lub zbyt zamknięte na modyfikacje. Budowa własnego rozwiązania pozwoliła uwzględnić aspekt ekonomiczny, oferując funkcjonalną i tańszą alternatywę dostosowaną do specyficznych wymagań użytkownika.

Realizacja pracy wymagała spełnienia szeregu założeń technicznych, w tym implementacji systemu operacyjnego czasu rzeczywistego (FreeRTOS – Free Real Time Operating System), w celu zapewnienia współbieżności procesów akwizycji i transmisji danych oraz stworzenia obudowy umożliwiającej testy w warunkach drogowych.

2. Cel i zakres pracy

Celem głównym jest zaprojektowanie i realizacja urządzenia monitorowania położenia pojazdu z wykorzystaniem mikrokontrolera ESP32.

2.1. Szczegółowe cele pracy

Niniejszy podrozdział precyzuje zakres prac niezbędnych do realizacji celu głównego. Poniżej przedstawione zostały cele szczegółowe, które stanowią fundament planowanych działań projektowych.

- Implementację oprogramowania, umożliwiającego akwizycję danych z modułu GNSS (Global Navigation Satellite System), ich archiwizację na karcie pamięci SD oraz zdalną synchronizację z serwerem.
- Zaprojektowanie i wykonanie warstwy sprzętowej urządzenia, w tym obwodu drukowanego (lub układu prototypowego) oraz obudowy.
- Przeprowadzenie testów funkcjonalnych prototypu w warunkach rzeczywistych.
- Analizę jakości pozyskanych danych lokalizacyjnych oraz ocenę skuteczności mechanizmu synchronizacji danych.

2.2. Zakres pracy

W celu realizacji zdefiniowanych powyżej celów pracy, ustalono zakres prac składających się z:

- analizy wymagań projektowych i przeglądu konkurencyjnych rozwiązań,
- opracowania schematu połączeń,
- realizacji fizycznego prototypu urządzenia (montażu podzespołów),
- implementacji oprogramowania sterującego dla układu ESP32,
- weryfikacja działania urządzenia,
- przygotowania wniosków i propozycji dalszych prac.

Układ pracy został podzielony na rozdziały odpowiadające etapom procesu projektowego. Rozdział pierwszy stanowi wstęp, wprowadzający w tematykę systemów telemetrycznych i Internetu Rzeczy. W rozdziale drugim zdefiniowano cel główny, cele szczegółowe oraz zakres realizowanej pracy. Rozdział trzeci stanowi studium teoretyczne, zawierające przegląd wykorzystanych technologii (m.in. GNSS, MQTT – Message Queuing Telemetry Transport) oraz analizę porównawczą rozwiązań dostępnych na rynku. W rozdziale czwartym określono zakres funkcjonalny projektowanego urządzenia, przedstawiono proces projektowania i implementacji systemu, obejmujący opis architektury sprzętowej, a także strukturę oprogramowania oraz integrację z platformą serwerową ThingsBoard. Opisano tu również fizyczne wykonanie prototypu. W końcowej części tego rozdziału przedstawiono wyniki przeprowadzonych testów elektrycznych, funkcjonalnych oraz drogowych. Rozdział piąty zawiera podsumowanie i wnioski. Omówiono w nim napotkane problemy i ograniczenia realizacyjne, sformułowano finalne wnioski z realizacji projektu oraz zaproponowano kierunki dalszego rozwoju systemu.

3. Podstawy teoretyczne i przegląd rozwiązań

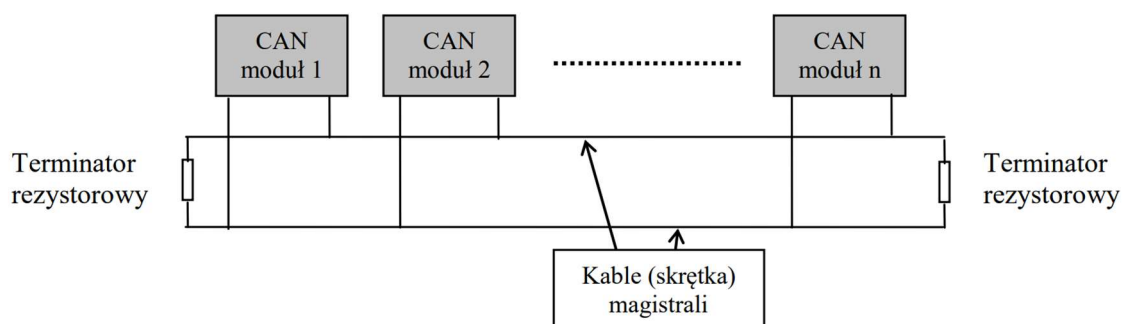
3.1. Przegląd technologiczny

Współczesne systemy telemetryczne i lokalizacyjne wymagają integracji wielu warstw technologicznych, począwszy od niskopoziomowej obsługi sprzętu, poprzez systemy operacyjne czasu rzeczywistego, aż po protokoły komunikacyjne. Niniejszy rozdział stanowi studium technologii wybranych do realizacji projektu inżynierskiego.

3.1.1. Magistrala CAN

Magistrala CAN to szeregową magistrala komunikacyjna opracowana przez firmę Bosch w latach 80. XX wieku, znormalizowana jako ISO 11898. Jest ona nadal jednym z kluczowych standardów wymiany danych w systemach motoryzacyjnych oraz automatyce przemysłowej [3].

CAN wykorzystuje transmisję różnicową, co zapewnia wysoką odporność na zakłócenia elektromagnetyczne, co jest cechą kluczową w środowisku pracy jakim jest pojazd. Dla sieci CAN pokazanej na rysunku 3.1, fizyczne medium transmisyjne stanowi zazwyczaj para skręconych przewodów tzw. „skrętka”, zakończona rezystorami terminującymi o wartości $120\ \Omega$, co zapobiega odbiciom sygnału.



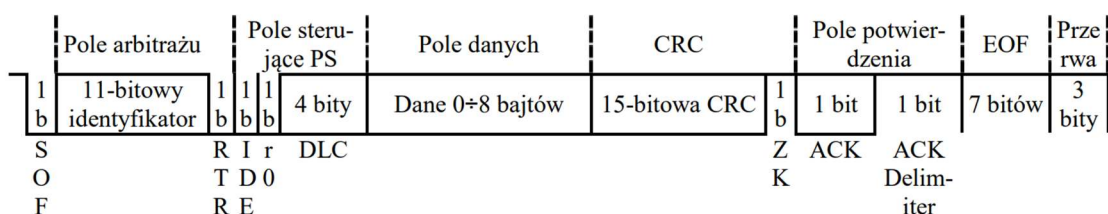
Rys. 3.1. Podstawowy schemat magistrali CAN [4]

Sygnał interpretowany jest na podstawie różnicy potencjałów między liniami CAN High i CAN Low [4]:

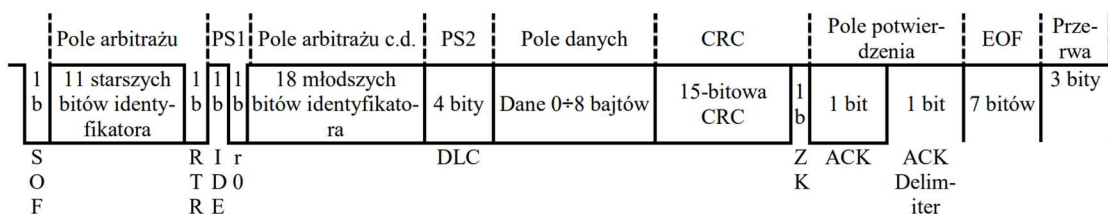
- **Stan recesywny (logiczna „1”)**: występuje, gdy napięcie na obu liniach jest zbliżone (ok. 2,5 V).
- **Stan dominujący (logiczne „0”)**: występuje, gdy różnica napięć wzrasta ($CAN\ High = 3,5\ V$, $CAN\ Low = 1,5\ V$).

W przypadku jednoczesnego nadawania przez dwa węzły, „wygrywa” ten, który nadaje stan dominujący („0”) na bitach identyfikatora o niższej wadze liczbowej (niższy numer ID oznacza wyższy priorytet).

Format ramek CAN pokazano na rysunkach 3.1 oraz 3.2.



Rys. 3.2 Struktura ramki danych (format ramki standardowej – CAN 2.0A) [5]



Rys. 3.3 Struktura ramki danych (format ramki rozszerzonej – CAN 2.0B) [5]

Zalety sieci CAN obejmują między innymi [5]:

- brak jednostki centralnej,
- duża prędkość transmisji,
- odporność na zakłócenia,
- sprawne procedury obsługi błędów,
- możliwość podłączenia dużej liczby stacji,
- możliwość utworzenia 2^{29} różnych identyfikatorów wiadomości.

3.1.2. Protokół MQTT

MQTT to protokół transmisji danych (charakteryzujący się niskim narzutem danych, gdzie nagłówek pakietu może wynosić zaledwie 2 bajty), wykorzystujący protokół TCP – Transmission Control Protocol / IP – Internet Protocol jako warstwę transportową. Został znormalizowany w 2014 roku przez firmę OASIS. Jest często stosowany w systemach IoT oraz teledyktacji pojazdu. [6]

W przeciwieństwie do modelu klient-serwer (znanego np. z HTTP – Hypertext Transfer Protocol), MQTT wprowadza trzeci podmiot – brokera wiadomości. Architektura składa się z trzech elementów [6]:

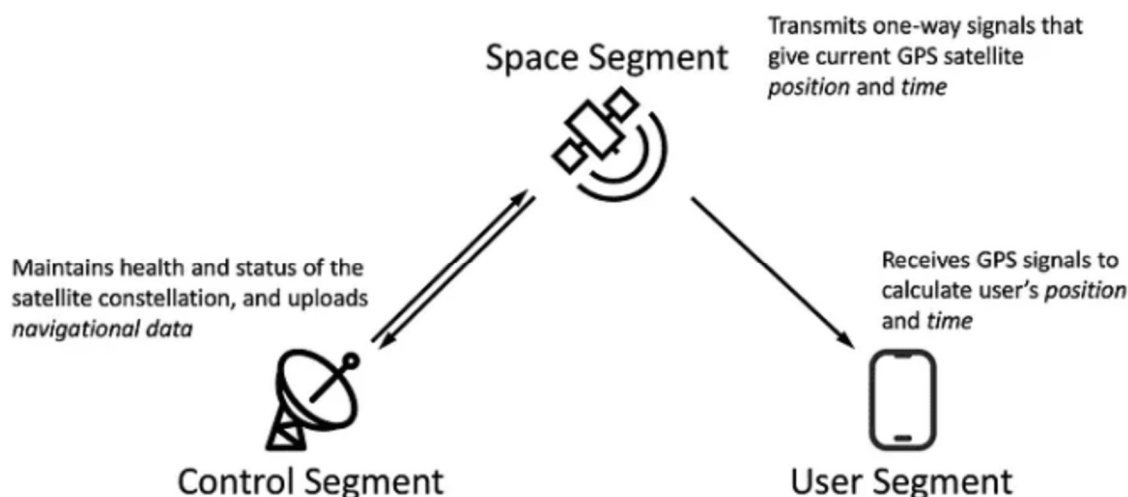
- **Wydawca (ang. Publisher):** urządzenie końcowe (lokalizator), które wysyła dane teledyktacyjne.
- **Broker:** serwer pośredniczący, odpowiedzialny za filtrowanie i dystrybucję wiadomości.
- **Subskrybent (ang. Subscriber):** aplikacja lub baza danych odbierająca informacje.

Taka architektura zapewnia asynchroniczność komunikacji oraz skalowalność systemu – wydawca nie musi znać adresu IP odbiorcy ani jego stanu aktywności. Protokół MQTT definiuje trzy poziomy jakości usług (ang. QoS – Quality of Service), które pozwalają dostosować pewność dostarczenia danych do warunków sieciowych (np. niestabilnego połączenia GSM – Global System for Mobile Communications) [6]:

- **QoS 0 (At most once):** „Wyślij i zapomnij”. Brak potwierdzenia odbioru. Najmniejsze zużycie transferu.
- **QoS 1 (At least once):** gwarancja dostarczenia (możliwe duplikaty). Wymagane potwierdzenie (PUBACK).
- **QoS 2 (Exactly once):** gwarancja jednokrotnego dostarczenia. Największy narzut komunikacyjny.

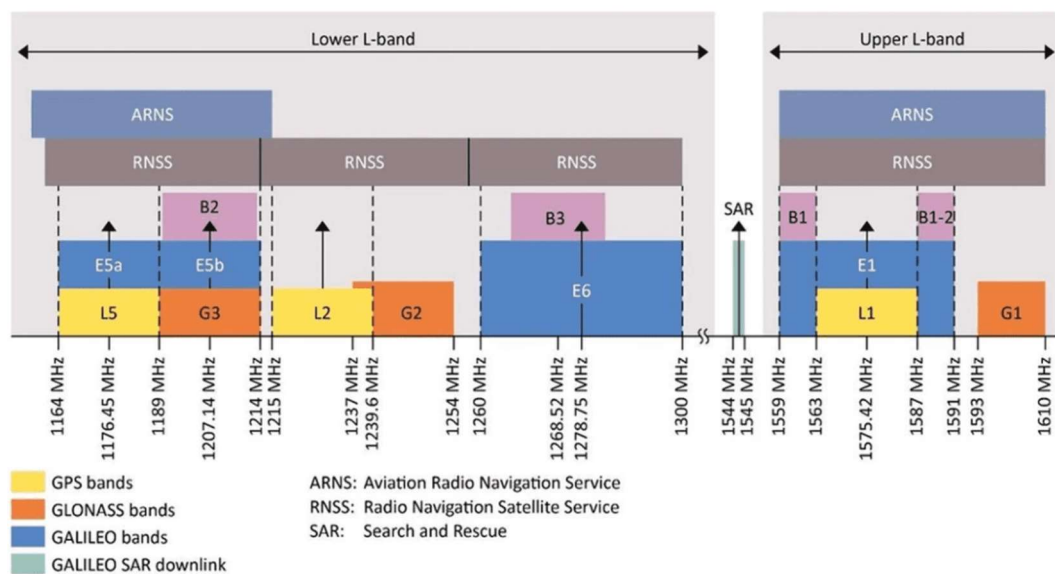
3.1.3. Systemy GNSS i standard NMEA 0183

GNSS to ogólna nazwa systemów nawigacji satelitarnej, obejmująca m.in. amerykański GPS, europejski Galileo, rosyjski GLONASS oraz chiński BeiDou. Architektura systemów GNSS składa się ze stacji naziemnych, satelitów oraz odbiorników końcowych, co można zaobserwować na rysunku 3.4.



Rys. 3.4. Architektura systemu GNSS. [7]

Większość tych satelitów operuje na średniej orbicie okołoziemskiej (ang. MEO – Medium Earth Orbit), czyli około 20000 km. Każdy z nich wysyła sygnał modulowany o odpowiedniej częstotliwości [7]. W zależności od systemu ta częstotliwość jest inna co można zaobserwować na rysunku 3.5.



Rys. 3.5. Pasma częstotliwościowe dla systemów GNSS. [7]

Zasada wyznaczania pozycji opiera się na trilateracji sferycznej, gdzie odbiornik oblicza swoją lokalizację na podstawie pomiaru czasu propagacji sygnału radiowego z co najmniej czterech satelitów [7].

Standard NMEA 0183 to dobrowolny standard specyfikacji interfejsu elektrycznego i protokołu danych dla sprzętu morskiego i nawigacyjnego, opracowany przez National Marine Electronics Association. W systemach wbudowanych komunikacja odbywa się zazwyczaj poprzez interfejs asynchroniczny UART (Universal Asynchronous Receiver/Transmitter). Dane przesyłane są w postaci czytelnych ciągów znaków ASCII, zwanych „zdaniami”.

Przykład struktury ramki RMC (Recommended Minimum Specific GNSS Data)[8]:

\$GPRMC,210230,A,3855.4487,N,09446.0071,W,0.0,076.2,130495,003.8,E*69

Tabela 3.1 Wyjaśnienie struktury ramki RMC

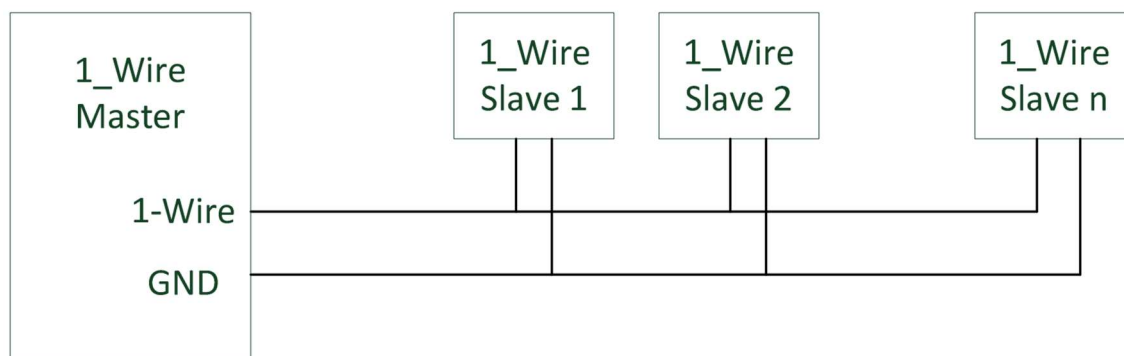
Przykład	Wyjaśnienie
\$	Początek sekwencji
GPRMC	Typ wiadomości
210230	Obecny czas UTC
A	Status pozycji (A - poprawny, V – niepoprawny)
3855.4487,N	Szerokość geograficzna wraz z kierunkiem (DDMM.MMM)
09446.0071,W	Długość geograficzna wraz z kierunkiem (DDDMM.MMM)
0.0	Prędkość
076.2	Kurs
130495	Data (DDMMYY)
003.8,E	Wariacja magnetyczna wraz z kierunkiem
*69	Suma kontrolna (w systemie szesnastkowym)

Technologia EASY™ (Embedded Assist System) jest istotnym rozszerzeniem funkcjonalności w nowoczesnych odbiornikach GNSS. Służy ona do skrócenia czasu ustalenia pozycji. W przeciwieństwie do rozwiązań typu A-GPS wymagających

połączenia z siecią, EASY działa autonomicznie – odbiornik oblicza i przewiduje orbity satelitów na okres do 3 dni naprzód, bazując na danych odebranych podczas ostatniej aktywności. Informacje te są przechowywane w podtrzymywanej bateryjnie pamięci RAM lub Flash. Dzięki temu, po ponownym uruchomieniu urządzenia (nawet przy braku bieżącego sygnału z satelitów przez pierwsze sekundy), moduł może błyskawicznie wyznaczyć pozycję, korzystając z zapamiętanych efemeryd [9].

3.1.4. Magistrala 1-Wire

1-Wire to system magistrali komunikacyjnej zaprojektowany przez Dallas Semiconductor (obecnie Analog Devices), który zapewnia stosunkowo niską prędkość transmisji danych na poziomie 16 kbps. Magistrala działa w układzie „Master-Slave”, co przedstawia rys.3.3.



Rys. 3.6. Podstawowy schemat sieci 1-Wire. [10]

Kluczową cechą magistrali jest wyjście typu otwarty dren z rezystorem podciągającym (o wartości 1000-5000 Ω) do zasilania (zazwyczaj 3,3 V lub 5 V). Umożliwia to realizację tzw. „zasilania pasożytniczego” – urządzenia podrzędne mogą czerpać energię z linii danych w stanie wysokim, ładując wewnętrzny kondensator, co pozwala na redukcję okablowania do dwóch przewodów (DATA, GND). [10]

Każde urządzenie 1-Wire posiada fabrycznie nadany, unikalny, 64-bitowy kod ROM, na który składa się [10]:

- 8 bitów kodu rodziny: identyfikuje typ urządzenia (np. termometr),
- 48 bitów unikalnego numeru seryjnego,
- 8 bitów sumy kontrolnej CRC.

3.1.5. FreeRTOS

FreeRTOS jest systemem czasu rzeczywistego zapewniającym determinizm czasowy, co oznacza że czas reakcji musi być znany i powtarzalny. Sercem tego systemu jest planista (ang. Scheduler), który odpowiada za wykonywanie zadania o aktualnie najwyższym priorytecie.

Aby zapewnić stabilną wymianę danych i synchronizację między wątkami, wykorzystuje się zaawansowane mechanizmy komunikacji międzyprocesowej (ang. Inter Process Communication) [11]:

- **Powiadomienia:** stosuje się w celu wysłania bezpośrednio powiadomienia do innego zadania,
- **Kolejki:** służą do bezpiecznego buforowania i przesyłania struktur danych między zadaniami,
- **Semafory:** są używane do synchronizacji i zarządzania zasobami,
- **Mutexy:** semafory, które posiadają mechanizm dziedziczenia priorytetów,
- **Grupy Zdarzeń:** pozwalają zadaniu czekać na kombinacje wielu flag bitowych.

Dodatkowo, nadzór nad stabilnością pracy sprawuje sprzętowy „Watchdog Timer”, który resetuje układ w przypadku wykrycia anomalii w działaniu któregośkolwiek z krytycznych wątków.

3.2. Analiza wybranych rozwiązań rynkowych

Rynek systemów lokalizacji pojazdów cechuje się obecnie szeroką gamą rozwiązań o zróżnicowanej funkcjonalności, cenie oraz przeznaczeniu. Od prostych rejestratorów tras, przez systemy antykradzieżowe, aż po zaawansowaną telematykę flotową. W celu osadzenia realizowanego projektu w kontekście rynkowym, dokonano analizy trzech reprezentatywnych urządzeń dostępnych w sprzedaży oraz porównano je z projektowanym rozwiązaniem.

Do przeglądu wybrano urządzenia reprezentujące różne segmenty rynku:

- klasyczny lokalizator montowany na stałe,

- rozwiązanie typu „Plug & Play” pod złącze OBD,
- przenośny lokalizator z własnym zasilaniem.

3.2.1. Lokalizator GPS BearPoint C18



Rys. 3.4. Urządzenie BearPoint C18. [12]

BearPoint C18 (rys. 3.4) stanowi przykład klasycznego lokalizatora montowanego na stałe w instalacji elektrycznej pojazdu, dedykowanego zarówno klientom indywidualnym, jak i zarządom flot. Warstwa komunikacyjna urządzenia bazuje na technologii 4G/LTE z mechanizmem automatycznego przełączania do sieci 2G, co gwarantuje stabilność zasięgu. Konstrukcja sprzętowa uwzględnia wbudowaną baterię, podtrzymującą pracę modułu w przypadku awarii lub sabotażu zasilania głównego. W zakresie funkcjonalności system oferuje śledzenie pozycji w czasie rzeczywistym, obsługę geostref, alerty o przekroczeniu prędkości oraz możliwość zdalnego unieruchomienia pojazdu poprzez wysterowanie przekaźnika zapłonu. Z perspektywy modelu biznesowego rozwiązanie to opiera się na udostępnieniu przez producenta gotowego środowiska wizualizacyjnego, co zazwyczaj wiąże się z modelem abonamentowym i generuje stałe koszty eksploatacji.

3.2.2. Lokalizator GPS OBD MK08



Rys. 3.5. Urządzenie MK08. [13]

MK08 (rys. 3.5) to przedstawiciel kategorii urządzeń „Plug & Play”, które wykorzystują złącze diagnostyczne OBDII jako źródło zasilania oraz punkt montażowy. Kluczową zaletą tego rozwiązania jest brak konieczności ingerencji w instalację elektryczną pojazdu, proces instalacji ogranicza się do wpięcia urządzenia w gniazdo diagnostyczne. W zakresie funkcjonalności moduł oferuje podstawowe usługi lokalizacyjne, archiwizację historii tras oraz system alarmów wstrząsowych. Warto jednak odnotować, że budżetowe wersje tego typu rozwiązań często wykorzystują interfejs OBDII wyłącznie do zasilania, nie implementując pełnej obsługi protokołu diagnostycznego (brak odczytu parametrów takich jak RPM czy kody błędów), co stanowi istotne ograniczenie względem zaawansowanych systemów telematycznych. Ciągłe zasilanie z instalacji pojazdu rozwiązuje problem konieczności ładowania baterii wewnętrznej, lecz w przypadku dłuższego postoju może prowadzić do nadmiernego obciążenia akumulatora rozruchowego.

3.2.3. Lokalizator GPS MK6B 4G



Rys. 3.6. Urządzenie MK6B. [14]

Model MK6B (rys. 3.6) reprezentuje segment lokalizatorów mobilnych, działających niezależnie od stałej instalacji elektrycznej pojazdu. Cechą wyróżniającą to rozwiązanie jest implementacja ogniwa o wysokiej pojemności (3000 mAh), co pozwala na osiągnięcie czasu pracy wynoszącego nawet 35 dni, w zależności od skonfigurowanej częstotliwości rejestracji i wysyłania danych. Urządzenie wyposażono w zintegrowany montaż magnetyczny, umożliwiający szybką instalację na zewnętrznych elementach karoserii lub kontenerach transportowych. Warstwa komunikacyjna oparta jest na modemie 4G LTE, zapewniającym transmisję danych w czasie rzeczywistym. Ze względu na swoją autonomiczność energetyczną, urządzenie to znajduje zastosowanie głównie w monitoringu doraźnym, zabezpieczaniu ładunków oraz śledzeniu maszyn budowlanych pozbawionych dostępu do pokładowej sieci zasilającej.

3.2.4. Kategoryzacja dostępnych rozwiązań

Na podstawie przeglądu rynku można wyróżnić trzy główne kategorie systemów lokalizacji, różniące się architekturą, kosztami, dokładnością i grupą docelową.

Rozwiązania OEM (Original Equipment Manufacturer): Systemy fabrycznie montowane w nowoczesnych pojazdach, głęboko zintegrowane z magistralą CAN i systemami multimedialnymi. Stawiają na wygodę użytkownika cywilnego, a nie na analizę logistyczną. Oferują najwyższą jakość danych diagnostycznych poprzez wykorzystanie m.in. eSIM (Embedded SIM), GPS, żyroskopów, akcelerometrów, kąta skrętu kół. Poprzez bezpośrednie wbudowanie w systemy pojazdy rozwiązania takie są zamknięte dla użytkownika. Większość producentów ogranicza system do podglądu pozycji parkowania ze względów prywatności. Koszty początkowe są zerowe (w cenie zakupu auta), i często abonament na usługi lokalizacyjne jest opłacony na 1-3 lat. Niestety po tym okresie trzeba zapłacić nawet kilkaset złotych rocznie za utrzymanie łączności.

Gotowe komercyjne lokalizatory GPS (np. BearPoint C18, MK08, MK6B): są to kompletne produkty, często zamknięte systemowo, zintegrowane z kartą SIM (Subscriber Identity Module) i dedykowaną chmurą producenta. Zapewniają wysoką niezawodność, dokładność oraz łatwość obsługi dla użytkownika końcowego. Koszt początkowy to zazwyczaj kilkaset złotych w zależności od producenta i modelu. Do tego dochodzą koszty abonamentu za kartę SIM oraz dostęp do chmury. Takie rozwiązanie wiąże się z brakiem możliwości modyfikacji oprogramowania czy sposobu przetwarzania danych.

Modułowe rozwiązania własnego wykonania „DIY” (ang. Do it yourself): są to systemy oparte na uniwersalnych mikrokontrolerach (ESP32, Arduino) i oddzielnych modułach GNSS/GSM. Koszt początkowy to wartość wszystkich komponentów potrzebnych do zbudowania urządzenia, zazwyczaj do kilkuset złotych. W zależności od zastosowanych technologii mogą również występować koszty abonamentu związane np. z kartą SIM. Rozwiązanie takie cechuje się wysoką elastycznością, relatywnie niskim kosztem początkowym i pełną kontrolą nad danymi, wymagają jednak samodzielnej integracji sprzętowej i programowej.

3.2.5. Wnioski z przeglądu i uzasadnienie wyboru technologii

Analiza konkurencyjnych rozwiązań prowadzi do następujących wniosków, które ukształtowały założenia projektowe niniejszej pracy:

- **Zasadność podejścia modułowego:** dla celów edukacyjnych i prototypowych, wykorzystanie mikrokontrolera jest wystarczające. Pozwala na obniżenie kosztów budowy urządzenia oraz umożliwia szybkie iteracje (zmiany w algorytmie i sprzęcie), co jest niemożliwe w zamkniętych produktach komercyjnych.
- **Akceptacja ograniczeń komunikacyjnych:** wybór Wi-Fi jako głównego kanału transmisji jest świadomym kompromisem pomiędzy ciągłością transmisji oferowaną przez GSM a minimalizacją kosztów, wynikającą z rezygnacji z modułów łączności (GSM) oraz abonamentu karty SIM. Choć komercyjne lokalizatory (BearPoint C18, MK08, MK6B) oferują ciągły monitoring przez LTE, w zastosowaniach takich jak analiza tras po fakcie, logistyka wewnętrzna czy „czarne skrzynki”, synchronizacja okresowa po powrocie do zasięgu znanej sieci jest wystarczająca.
- **Potencjał integracji OBD2:** przegląd urządzenia MK08 wskazuje na duży potencjał złącza diagnostycznego. Jednakże, aby w projekcie „DIY” uzyskać funkcjonalność wykraczającą poza samo zasilanie, konieczne jest zastosowanie dodatkowego transceivera CAN (np. TJA1050).

Podsumowując, realizowany projekt nie stanowi bezpośredniej konkurencji dla systemów antykradzieżowych (wymagających LTE), lecz jest elastyczną platformą telemetryczną, oferującą funkcjonalności niedostępne w tanich lokalizatorach, takie jak pełny dostęp do surowych danych i możliwość dowolnej konfiguracji parametrów zapisu.

4. Projekt i implementacja urządzenia

Rozdział ten opisuje szczegóły projektowe warstwy sprzętowej i programowej, prezentując sposób, w jaki omówione wcześniej technologie zostały zintegrowane w spójny system.

4.1. Założenia projektowe

Na podstawie zdefiniowanego celu głównego przyjęto szereg założeń funkcjonalnych, sprzętowych oraz programowych, które determinują sposób realizacji projektu.

4.1.1. Założenia funkcjonalne

Określają one zakres operacji realizowanych przez system, aby spełnić cel projektu. Poniżej zdefiniowano kluczowe funkcje i zachowania, jakimi musi charakteryzować się gotowe urządzenie.

- **Autonomia działania:** urządzenie powinno działać w sposób bezobsługowy, automatycznie rozpoczynając akwizycję danych po włączeniu zasilania.
- **Rejestracja danych:** system musi umożliwiać ciągły zapis parametrów lokalizacyjnych (szerokość i długość geograficzna, czas, prędkość) na lokalnym nośniku danych (karta SD).
- **Synchronizacja danych:** urządzenie powinno posiadać mechanizm wykrywania dostępności połączenia sieciowego i automatycznego przesyłania zgromadzonych danych na serwer zewnętrzny (buforowanie danych w trybie offline).
- **Sygnalizacja stanu:** zastosowanie diod LED (Light Emitting Diode) lub innego interfejsu informującego użytkownika o statusie pracy (np. status WiFi, status karty SD, aktywność GNSS).
- **Możliwość rozbudowy:** projekt powinien uwzględniać możliwość przyszłego dołączenia interfejsu diagnostycznego pojazdu (OBD/CAN) oraz dodatkowych czujników (np. temperatury).

4.1.2. Założenia sprzętowe

Definiują one wymagania dotyczące fizycznej konstrukcji urządzenia, w tym doboru układów elektronicznych i parametrów zasilania. Do realizacji projektu niezbędne jest zastosowanie podzespołów spełniających następujące kryteria.

- **Jednostka sterująca:** wykorzystanie mikrokontrolera z wbudowanym modulem komunikacji bezprzewodowej (Wi-Fi/Bluetooth) oraz wysoką wydajnością obliczeniową.
- **Moduł lokalizacyjny:** zastosowanie modułu GNSS obsługującego standard NMEA, zapewniającego dokładność lokalizacji wystarczającą do śledzenia trasy pojazdu.
- **Zasilanie:** system przystosowany do zasilania z instalacji samochodowej (poprzez odpowiedni układ obniżający napięcie) lub zewnętrznego źródła mobilnego (powerbank/akumulator Li-Ion).
- **Pamięć masowa:** wykorzystanie karty pamięci SD jako bufora danych o dużej pojemności, umożliwiającego długotrwały zapis historii tras.
- **Obudowa:** konstrukcja mechaniczna powinna zapewniać ochronę układów elektronicznych przed uszkodzeniami mechanicznymi.

4.1.3. Założenia programowe

Dotyczą architektury kodu, wyboru języka programowania, środowiska deweloperskiego oraz zastosowanych algorytmów i protokołów komunikacyjnych. Warstwę logiczną zaprojektowano w oparciu o następujące wytyczne i narzędzia:

- **Środowisko programistyczne:** oprogramowanie zostanie zrealizowane w języku C++ w oparciu o środowisko programistyczne Arduino, co zapewni dostęp do szerokiej bazy bibliotek obsługi układów peryferyjnych.

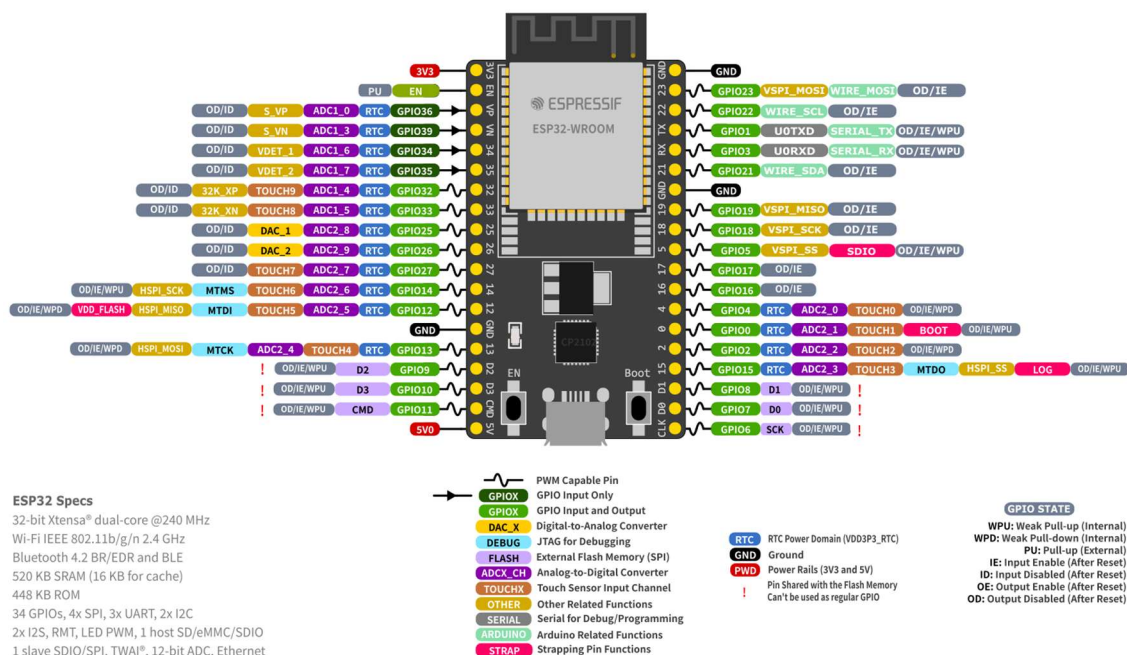
- **Struktura danych:** dane na karcie SD będą zapisywane w ustrukturyzowanym formacie tekstowym (np. CSV – Comma Separated Values, JSON – JavaScript Object Notation), co umożliwi ich późniejsze przetwarzanie i analizę.
- **Obsługa błędów:** system musi być odporny na typowe błędy, takie jak brak karty SD, utrata sygnału GNSS czy brak połączenia z serwerem, i wznowiać pracę po ustąpieniu awarii.

4.2. Architektura sprzętowa

Zaprojektowany system akwizycji danych opiera się na architekturze modułowej, w której jednostka centralna zarządza układami peryferyjnymi odpowiedzialnymi za pozycjonowanie, komunikację z magistralą pojazdu, pomiary środowiskowe oraz archiwizację danych. Poniżej przedstawiono szczegółową charakterystykę poszczególnych bloków funkcjonalnych urządzenia.

4.2.1. Jednostka centralna

Jako główny sterownik systemu wybrano moduł deweloperski ESP32-DevKitC V4 (widocznym na rysunku 4.1) firmy Espressif Systems. Decyzja ta podyktowana była obecnością dwurdzeniowego procesora, który umożliwia jednoczesną obsługę stosu komunikacyjnego Wi-Fi oraz zadań czasu rzeczywistego (akwizycja danych z czujników). Mikrokontroler pracuje w standardzie napięciowym 3,3 V i dysponuje wystarczającą liczbą interfejsów sprzętowych (UART, SPI, TWAI/CAN) do obsługi wszystkich założonych modułów bez konieczności stosowania ekspanderów wyprowadzeń. [15]



Rys. 4.1. Schemat wyprowadzeń ESP32-DevKitC V4. [15]

4.2.2. Układy peryferyjne

1. Moduł pozycjonowania GNSS

Do określania pozycji geograficznej wykorzystano układ Quectel LC76G. Jest to odbiornik Multi-GNSS obsługujący systemy GPS, GLONASS, Galileo oraz BeiDou, co zapewnia wysoką precyzję i szybkość ustalania pozycji nawet w trudnych warunkach terenowych. Komunikacja z mikrokontrolerem odbywa się za pośrednictwem interfejsu UART (piny GPIO 16 i 17), przy prędkości transmisji 9600 bps. Dodatkowo, system wykorzystuje sygnał PPS (Pulse Per Second) podłączony do pinu GPIO 32 w celu precyzyjnej synchronizacji czasu systemowego. [16]

2. Komunikacja z magistralą CAN

W celu odczytu danych telemetrycznych z pojazdu (np. prędkość pojazdu), zastosowano zewnętrzny transceiver CAN-PAL oparty na układzie TJA1051T/3. Pełni on rolę medium fizycznego, konwertując poziomy napięcie magistrali różnicowej CAN na poziomy logiczne akceptowane przez sterownik CAN (TWAI) wbudowany w układ ESP32. Transceiver podłączono do pinów GPIO 21 (RX) oraz GPIO 22 (TX). [17]

3. Pomiar temperatury

Do pomiaru temperatury wykorzystano cyfrowy czujnik temperatury DS18B20, wykorzystujący magistralę 1-Wire o parametrach technicznych:

- **Zakres pomiarowy:** od $-55\text{ }^{\circ}\text{C}$ do $+100\text{ }^{\circ}\text{C}$. (ograniczenie do $100\text{ }^{\circ}\text{C}$ wynika z wytrzymałości przewodów.)
- **Dokładność:** $\pm 0,5\text{ }^{\circ}\text{C}$ (w zakresie od $-10\text{ }^{\circ}\text{C}$ do $+85\text{ }^{\circ}\text{C}$).
- **Rozdzielczość:** od 9 do 12 bitów.

Został on podpięty do GPIO 4 co pozwoliło na monitorowanie temperatury otoczenia lub, w zależności od montażu, temperatury wewnątrz pojazdu. [18]

4. Pamięć masowa

W celu zapewnienia redundancji danych oraz możliwości pracy w trybie offline, system wyposażono w moduł czytnika kart microSD komunikujący się poprzez magistralę SPI (piny GPIO 5, 18, 19, 23). Umożliwia to zapis danych do archiwizacji lub buforowanie danych w przypadku braku połączenia z siecią Wi-Fi.

4.2.3. Układ zasilania

Ze względu na specyfikę pracy w pojeździe, zaprojektowano hybrydowy układ zasilania:

- **Wejścia zasilania:** urządzenie może być zasilane z instalacji 12 V pojazdu (poprzez złącze OBDII) lub z portu USB-C.
- **Przetwornica Step-Down:** napięcie 12 V jest obniżane do 5 V za pomocą przetwornicy impulsowej.
- **Zabezpieczenie przetwornicy:** zastosowano diodę Schottky'ego co pozwala na bezprzerwowe przełączanie między zasilaniem USB-C lub 12 V z instalacji pojazdu.
- **Zasilanie awaryjne (UPS):** ogniwo Li-Ion typu 18650 (Panasonic NCR18650B) wraz z modułem ładowania i przetwornicą podwyższającą, która zapewnia ciągłość pracy urządzenia po wyłączeniu zapłonu w pojeździe.

4.3. Architektura oprogramowania

Oprogramowanie urządzenia zaimplementowano w języku C++ z wykorzystaniem frameworka Arduino dla platformy ESP32. Ze względu na konieczność równoległej obsługi wielu procesów asynchronicznych, takich jak akwizycja danych z czujników, obsługa stosu sieciowego TCP/IP, operacje plikowe oraz obsługa żądań HTTP, zrezygnowano z klasycznej pętli głównej (super-loop) na rzecz systemu operacyjnego czasu rzeczywistego FreeRTOS.

4.3.1. Organizacja kodu źródłowego i struktura plików

Kod źródłowy projektu został przygotowany w środowisku Visual Studio Code z wykorzystaniem Arduino IDE jako narzędzia do wgrywania kodu na ESP32. Zarządzanie cyklem życia oprogramowania oraz historią zmian realizowano przy użyciu rozproszonego systemu kontroli wersji Git. Kompletne repozytorium projektu, zawierające pełną strukturę katalogów, pliki konfiguracyjne oraz dokumentację techniczną, zostało umieszczone w serwisie hostingowym GitHub. Takie podejście umożliwia łatwą replikację środowiska programistycznego oraz weryfikację poszczególnych etapów powstawania oprogramowania. Kod źródłowy podzielono na moduły funkcjonalne zgodnie z zasadami programowania obiektowego. Poniżej przedstawiono charakterystykę kluczowych plików projektu:

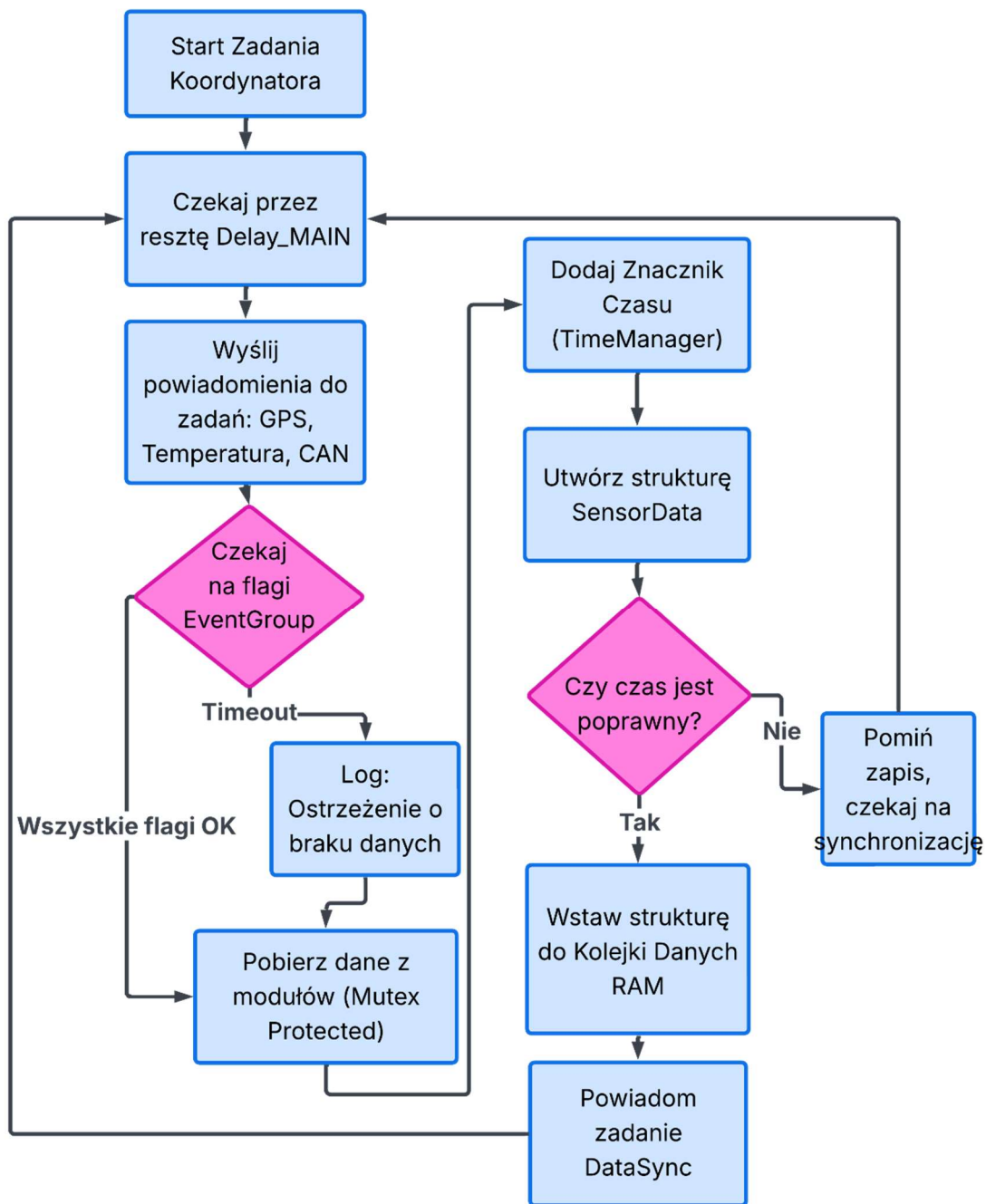
- **main.ino:** zawiera konfigurację sprzętową (setup()), inicjalizację obiektów systemowych (kolejki, semaforey) oraz implementację głównego zadania CoordinatorTask,
- **SensorData.h:** plik nagłówkowy definiujący globalną strukturę danych oraz mapowania dla serializacji JSON (X-Macro). Jest to jedyne miejsce definicji parametrów telemetrycznych, co gwarantuje spójność w całym projekcie,
- **SdModule:** zaawansowany menedżer pamięci masowej. Odpowiada za fizyczny zapis na kartę SD i rotację plików archiwalnych,
- **ThingsBoardClient:** obsługuje protokół MQTT dla platformy Thingsboard, w tym autoryzację, przesyłanie telemetry oraz odbiór atrybutów zdalnej konfiguracji,

- **WiFiManager:** moduł zarządzający łącznością. Implementuje logikę skanowania i wyboru najlepszego punktu dostępowego z listy znanych sieci,
- **WebServerModule:** obsługuje lokalny serwer HTTP, udostępniając interfejs mapy,
- **GpsModule / CanModule:** klasy warstwy abstrakcji sprzętowej. Ukrywają one złożoność komunikacji UART/TWAI, udostępniając wyższym warstwom gotowe, przetworzone dane (np. prędkość w km/h, współrzędne geograficzne),
- **TimeManager:** moduł odpowiedzialny za synchronizację czasu. Zarządza priorytetami źródeł czasu (GPS > NTP > RTC) i dostarcza spójny znacznik czasowy Unix Timestamp dla wszystkich logowanych zdarzeń.

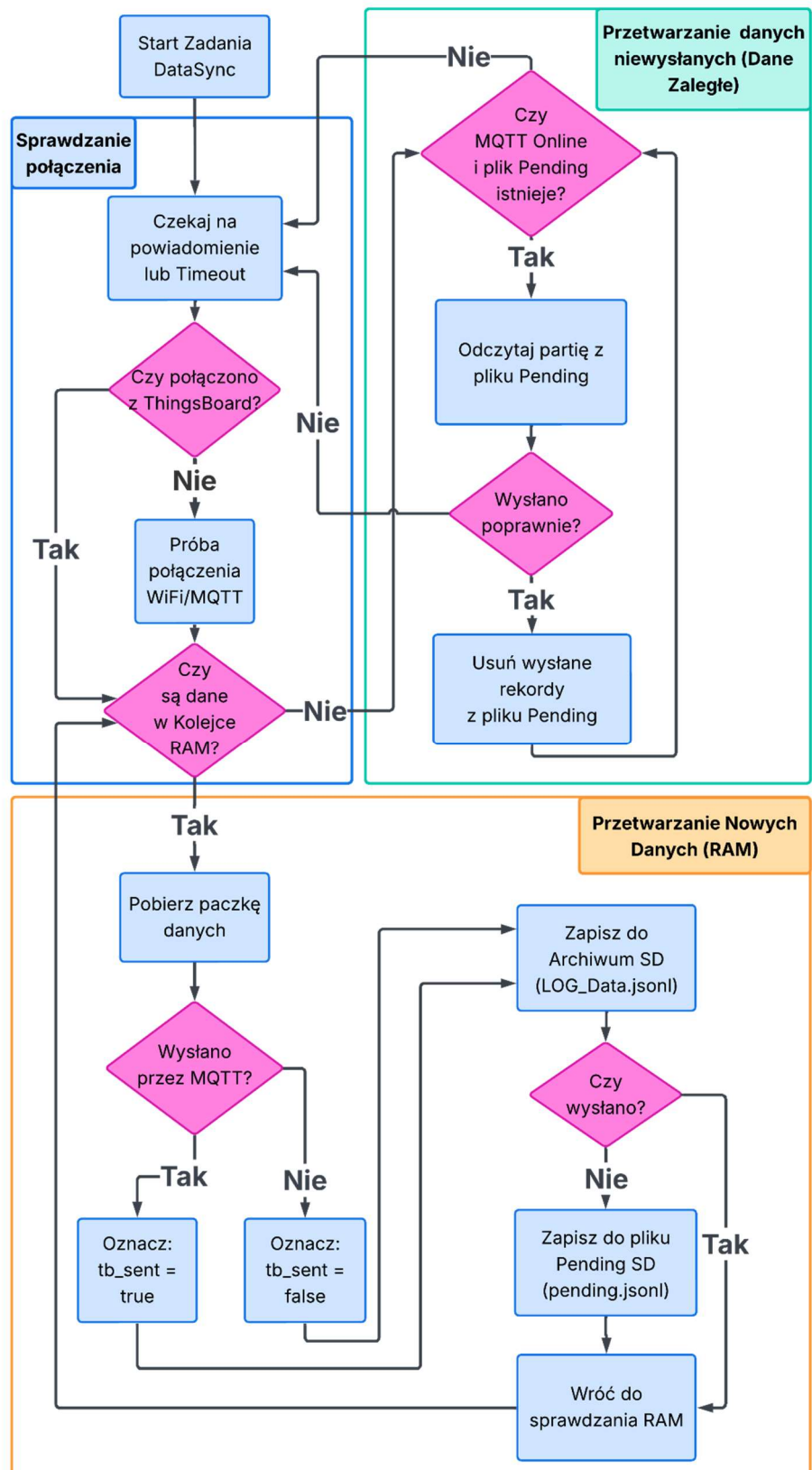
4.3.2. Podział na zadania

System operacyjny zarządza pulą niezależnych wątków o zróżnicowanych priorytetach:

- **CoordinatorTask:** cykliczne wyzwalanie pomiarów i agregacja danych (Snapshot),
- **TaskGPS / TaskTemp / TaskCAN:** obsługa poszczególnych czujników i protokołów komunikacyjnych,
- **TaskDataSync:** krytyczny proces odpowiedzialny za opróżnianie bufora danych (wysyłka MQTT oraz zapis SD),
- **TaskWiFi:** monitorowanie stanu połączenia i automatyczna reasocjacja,
- **TaskWebServer:** obsługa żądań HTTP od klientów lokalnych (przeglądarka),
- **TaskSleep:** zarządzanie energią i procesem bezpiecznego wyłączania urządzenia.



Rys. 4.2. Diagram funkcjonalny zadania koordynatora



Rys. 4.3. Diagram funkcjonalny zadania synchronizacji danych

4.3.3. Użyte biblioteki

Efektywna realizacja oprogramowania wbudowanego wymagała wykorzystania sprawdzonych bibliotek open-source, które zapewniają stabilność działania oraz abstrakcję warstwy sprzętowej. Poniżej zestawiono kluczowe biblioteki użyte w projekcie wraz z uzasadnieniem ich doboru.

- **ArduinoJson** [19]: jest to standard w środowisku Arduino służący do manipulacji danymi w formacie JSON. W projekcie biblioteka ta pełni kluczową rolę w formowaniu pakietów telemetrycznych wysyłanych do brokera MQTT. Pozwala na dynamiczne tworzenie struktur danych, co jest niezbędne przy zmiennej liczbie parametrów.
- **PubSubClient** [20]: biblioteka implementująca klienta protokołu MQTT. Została wybrana ze względu na swoją niezawodność, minimalne zużycie pamięci oraz pełną obsługę poziomów Quality of Service. Odpowiada za utrzymywanie połączenia z brokerem, obsługę mechanizmu „Keep-Alive” oraz publikowanie i subskrybowanie tematów.
- **TinyGPS++** [21]: biblioteka służąca do parsowania surowego strumienia danych NMEA 0183, odbieranego z modułu GNSS przez interfejs UART. Jej obiektowa struktura pozwala na łatwą ekstrakcję kluczowych informacji, takich jak szerokość i długość geograficzna, wysokość n.p.m., prędkość oraz czas, eliminując konieczność ręcznego analizowania ciągów znaków.
- **OneWire** [22] oraz **DallasTemperature** [23]: zestaw bibliotek obsługujący komunikację na magistrali 1-Wire. OneWire realizuje niskopoziomową obsługę protokołu, natomiast DallasTemperature dostarcza wysokopoziomowy interfejs do obsługi cyfrowych czujników temperatury z rodziny DS18B20, umożliwiając ich adresowanie i odczyt w trybie pasywnym.
- **ESP32-TWAI-CAN** [24]: biblioteka stanowiąca nakładkę dla wbudowanego w układ ESP32 kontrolera TWAI (Two Wire Automotive Interface), kompatybilnego ze standardem CAN 2.0B. Umożliwia ona filtrowanie ramek sprzętowych oraz obsługę przerwań, co jest kluczowe dla płynnego odczytu danych z magistrali pojazdu bez blokowania głównego wątku procesora.

4.3.4. Format danych i technika X-Macro

System wykorzystuje format JSON, jednak jego struktura różni się w zależności od medium docelowego:

Format chmurowy (ThingsBoard) [25]: Dane pomiarowe zgrupowane są w obiekcie „values”, co jest wymagane przez API (Application Programming Interface) platformy:

```
[
  {
    "ts": 1767307224811,
    "values": {
      "ts_source": 2,
      "lat": 50.0691678,
      "lon": 19.9044705,
      "alt": 220.32,
      "vel": 0.87044,
      "temp": 18.8125,
      "can_vel": 0,
      "ec": 0,
      "rssi": -36
    }
  },
  {
    "ts": 1767307224811,
    "values": {
      "ts_source": 2,
      "lat": 50.0691678,
      "lon": 19.9044705,
      "alt": 220.32,
      "vel": 0.87044,
      "temp": 18.8125,
      "can_vel": 0,
      "ec": 0,
      "rssi": -36
    }
  }
]
```

Format lokalny (Karta SD): Stosowany jest format JSON Lines, gdzie każdy wiersz stanowi płaski obiekt JSON:

```
{
  "ts":1767307224811,
  "ts_source":2,
  "lat":50.0691678,
  "lon":19.9044705,
  "alt":220.32,
  "vel":0.87044,
  "temp":18.8125,
  "can_vel":0,
  "lgr_ts":1767307224044,
  "lcr_ts":1767307224811,
  "lcr_ts":0,
  "ec":0,
  "tb_sent":true,
  "rssi":-36
}
```

W celu zapewnienia ścisłej spójności między strukturą danych w pamięci RAM mikrokontrolera a formatami wyjściowymi (JSON), w pliku nagłówkowym SensorData.h zastosowano zaawansowaną technikę preprocesora C/C++ zwaną X-Macro. Pozwala ona na zdefiniowanie listy pól danych w jednym centralnym miejscu, co eliminuje konieczność wielokrotnego definiowania tych samych struktur kodu i zmniejsza ryzyko błędów.

Poniżej przedstawiono fragment definicji makra mapującego:

```
#define SENSOR_DATA_MAP(XX) \
    XX(uint64_t, ts, "ts", false, true) \
    XX(double, lat, "lat", true, true) \
    /* ... kolejne definicje pól ... */
```

Makro to jest rozwijane dwukrotnie w procesie kompilacji, pełniąc dwie odrębne funkcje:

1. do budowy struktury SensorData w pamięci urządzenia,
2. do generowania funkcji sensorDataToTb oraz sensorDataToSd, które mapują pola struktury na odpowiednie klucze JSON.

Zastosowanie tej techniki eliminuje ryzyko powszechnych błędów programistycznych, takich jak pominięcie nowo dodanego pola w procedurze serializacji.

4.3.5. Zaawansowana obsługa pamięci

Moduł „SdModule” implementuje mechanizmy bezpieczeństwa i organizacji danych:

- **Automatyczna rotacja archiwum:** dane historyczne zapisywane są w plikach z datą w nazwie. Po przekroczeniu 5 MB tworzony jest nowy plik, co ułatwia późniejszą analizę danych oraz zarządzanie pamięcią masową..
- **Sprawdzanie karty:** przed każdym zapisem weryfikowana jest obecność karty SD. W razie błędu moduł podejmuje próbę ponownej inicjalizacji magistrali SPI.
- **Buforowanie danych:** obsługuje mechanizm tymczasowego przechowywania danych, które nie zostały jeszcze wysłane do chmury/serwera.

- **Archiwizacja danych pomiarowych:** realizuje stały zapis wszystkich odczytów z czujników do plików na karcie SD, używając formatu JSONL, co pozwala na zachowanie historii pomiarów niezależnie od transmisji sieciowej.

4.3.6. Mechanizmy synchronizacji i komunikacji międzyprocesowej

W celu zapewnienia integralności danych współdzielonych pomiędzy wątkami zastosowano mechanizmy synchronizacji dostępne w FreeRTOS:

- **Grupy zdarzeń (ang. Event Groups):** wykorzystywane przez Koordynatora do oczekiwania na zakończenie pracy przez wszystkie aktywne czujniki (EVENT_GPS_READY, EVENT_TEMP_READY, EVENT_CAN_READY). Pozwala to na synchronizację pomiarów, mimo różnego czasu ich trwania dla poszczególnych sensorów.
- **Kolejki (ang. Queues):** do przekazywania danych między Koordynatorem a zadaniem synchronizacji (TaskDataSync) zastosowano kolejkę „dataQueue” działającą jako bufor kołowy. Zapobiega to utracie próbek w sytuacjach, gdy zapis na kartę SD lub wysyłka sieciowa ulegną chwilowemu opóźnieniu.
- **Muteksy (ang. Mutexes):** zabezpieczają dostęp do współdzielonych zasobów sprzętowych, takich jak magistrala SPI (dla karty SD) oraz struktura danych pomiarowych, eliminując ryzyko wyścigów (ang. Race Conditions).

4.3.7. Zarządzanie danymi i tryb Offline

System implementuje hybrydowy model zapisu danych, zapewniający ciągłość historii pomiarowej niezależnie od stanu połączenia sieciowego:

- **Archiwizacja ciągła:** Każdy pomiarowy pakiet danych jest bezwarunkowo zapisywany w lokalnym archiwum na karcie SD w formacie JSON Lines (.jsonl). Pliki są rotowane automatycznie na podstawie rozmiaru.
- **Kolejkowanie Offline (Pending):** W przypadku braku połączenia z serwerem MQTT, dane trafiają dodatkowo do pliku tymczasowego (pending.jsonl). Po odzyskaniu połączenia, zadanie TaskDataSync wysyła zaległe pakiety danych, a następnie usuwa je z bufora lokalnego.

4.3.8. Synchronizacja czasu

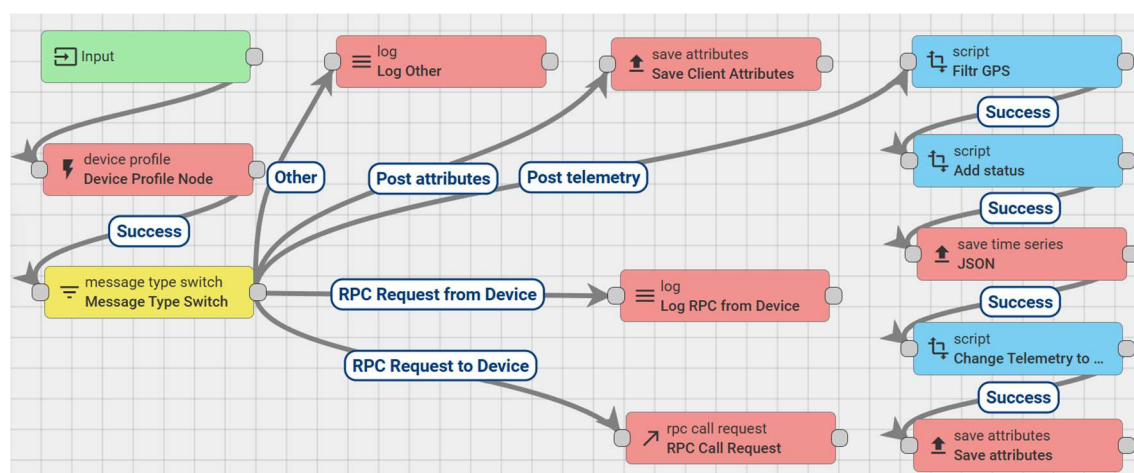
Kluczowym elementem systemu jest moduł TimeManager, który utrzymuje spójny czas systemowy (Unix Timestamp). System priorytetowo synchronizuje się z precyzyjnym sygnałem GPS, a w przypadku jego braku płynnie przechodzi na czas pobrany z sieci Wi-Fi (jeśli dostępny) lub wewnętrzny licznik RTC, gwarantując chronologiczną spójność logowanych danych.

4.4. Integracja z platformą ThingsBoard

Integralną częścią systemu jest warstwa serwerowa, zrealizowana w oparciu o platformę IoT ThingsBoard. Pełni ona rolę brokera MQTT, silnika reguł (ang. Rule Engine) oraz interfejsu użytkownika (ang. Dashboard). Rozwiązanie to pozwala na centralizację danych pomiarowych, ich wizualizację oraz zdalne zarządzanie parametrami pracy urządzenia bez konieczności fizycznej ingerencji w oprogramowanie układowe.

4.4.1. Przetwarzanie danych i Silnik Reguł

Algorytm przetwarzania przychodzących komunikatów MQTT został zdefiniowany za pomocą graficznego edytora reguł (ang. Rule Chain). Strumień danych z urządzenia trafia do węzła wejściowego, gdzie następuje jego klasyfikacja.

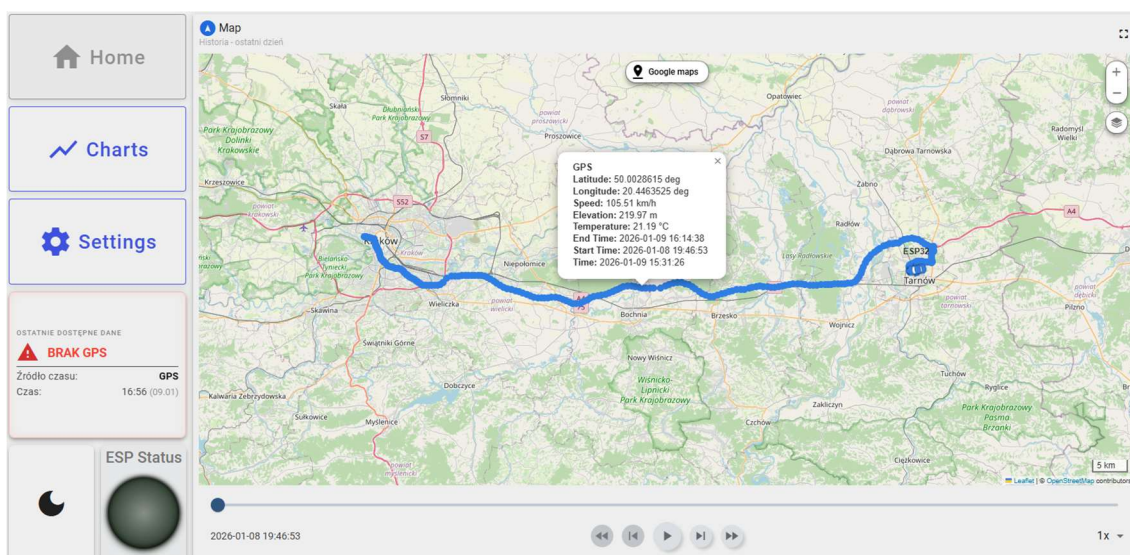


Rys. 4.4. Schemat łańcucha reguł skonfigurowanego na serwerze. Widoczne węzły odpowiedzialne za filtrację danych GPS, zapis telemetry oraz aktualizację atrybutów.

Jak przedstawiono na rysunku 4.4, system automatycznie rozdziela komunikaty na telemetrię (dane pomiarowe) oraz atrybuty (statusy). Zastosowano dedykowane skrypty przetwarzające (Nodes), m.in. Filtr GPS, który wstępnie weryfikuje poprawność współrzędnych przed ich zapisem do bazy danych, oraz skrypty aktualizujące status aktywności urządzenia.

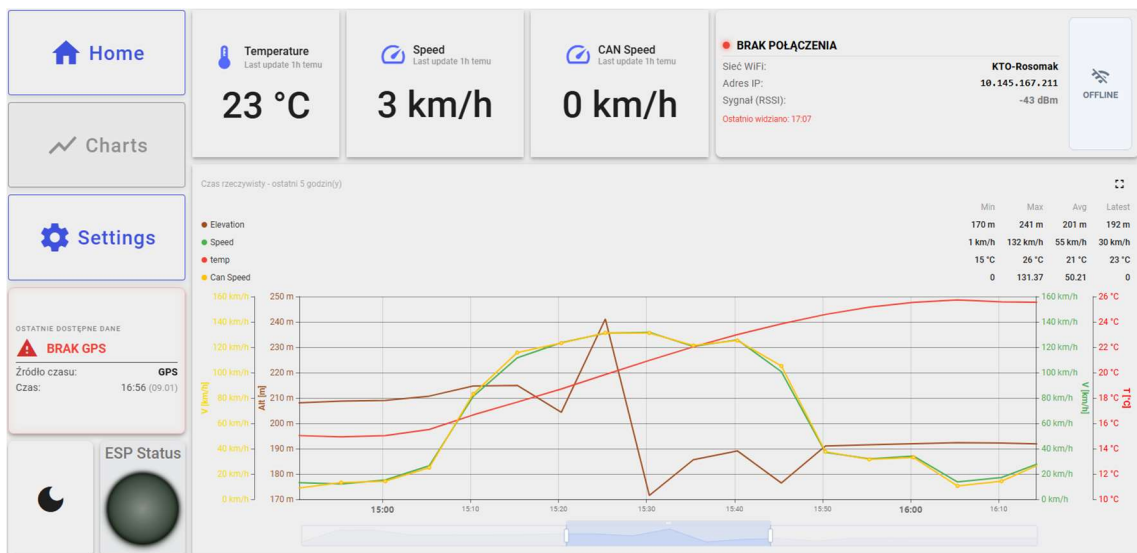
4.4.2. Wizualizacja danych i monitoring

Interfejs użytkownika został zaprojektowany w formie interaktywnego pulpitu (ang. Dashboard), podzielonego na sekcje tematyczne. Sekcja mapy (rysunek 4.5) wykorzystuje element interfejsu „Route Map”, który na podkładzie OpenStreetMap wizualizuje aktualną pozycję pojazdu oraz historię przebytej trasy. Interfejs umożliwia odtwarzanie przebiegu trasy w czasie (funkcja Time Window), co pozwala na analizę dynamiki przemieszczania się obiektu.



Rys. 4.5. Wizualizacja trasy przejazdu na mapie cyfrowej. Widoczny znacznik ostatniej pozycji oraz ślad historii przemieszczania.

Do monitorowania parametrów bieżących przygotowano panel „Charts” (rysunek 4.6). Zawiera on wskaźniki cyfrowe prezentujące prędkościomierz, temperaturę otoczenia, poziom sygnału Wi-Fi (RSSI) oraz status połączenia. Wykresy liniowe u dołu ekranu pozwalają na korelację parametrów w czasie, np. wpływu wzniesień terenu na prędkość pojazdu.



Rys. 4.6. Panel telemetryczny prezentujący parametry bieżące (prędkość, temperatura) oraz status diagnostyczny połączenia sieciowego.

4.4.3. Dwukierunkowa komunikacja i zdalna konfiguracja

Kluczową funkcjonalnością wyróżniającą zaprojektowany system jest możliwość zdalnej rekonfiguracji parametrów pracy poprzez mechanizm atrybutów współdzielonych (ang. Shared Attributes). W sekcji „Settings” (rysunek 4.7) przygotowano formularz, który pozwala użytkownikowi na modyfikację zmiennych systemowych.

The settings page includes the following fields and options:

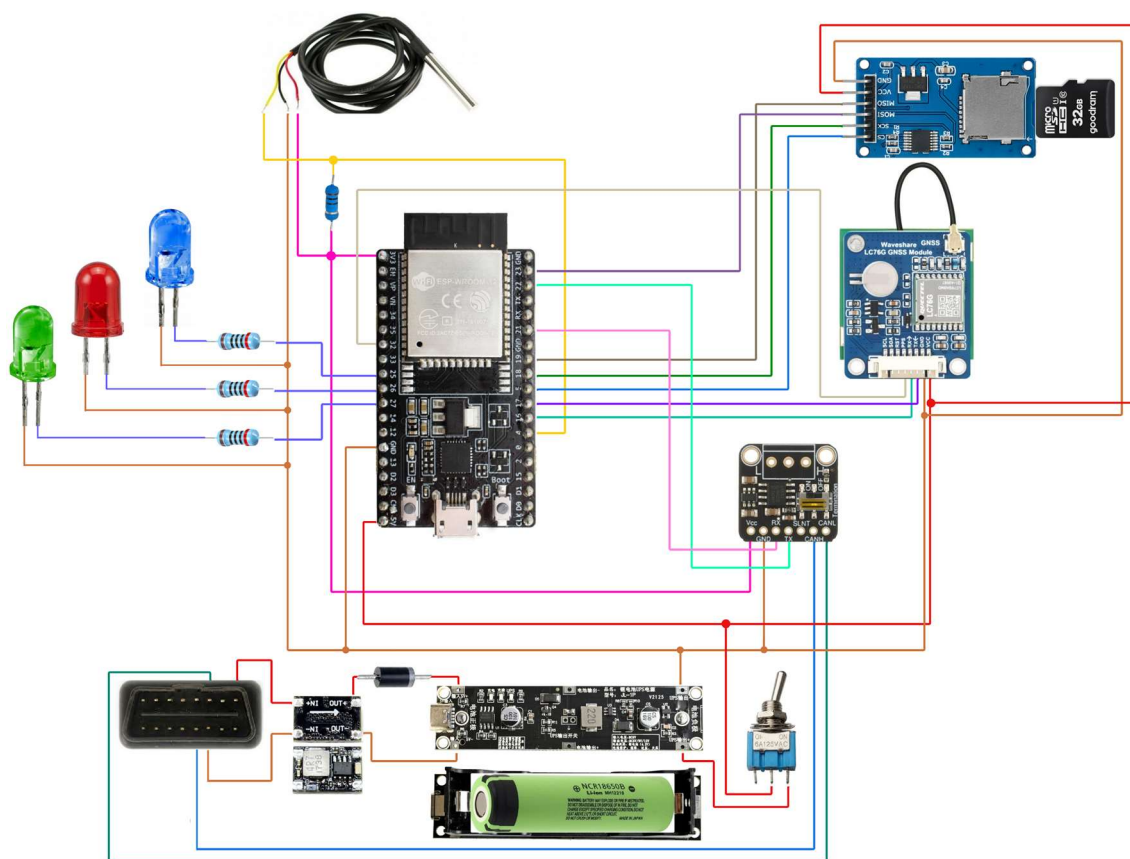
- Navigation:** Home, Charts, Settings.
- Status:**
 - OSTATNIE DOSTĘPNE DANE: **BRAK GPS**
 - Zródło czasu: GPS, Czas: 16:56 (09.01)
 - ESP Status:
- Configuration Fields:**
 - ESP32: Number of records sent in one package* (Value: 2)
 - Minimal Batch size to trigger save* (Value: 2)
 - Delay between data acquisition (s)* (Value: 15)
 - Delay between WiFi reconnections (s)* (Value: 60)
 - ☒ Require synchronized time
- Buttons:** Reset, Save

Rys. 4.7. Panel zdalnej konfiguracji urządzenia. Zmiana wartości w polach formularza skutkuje natychmiastową aktualizacją atrybutów wspólnych, które trafiają do ESP32.

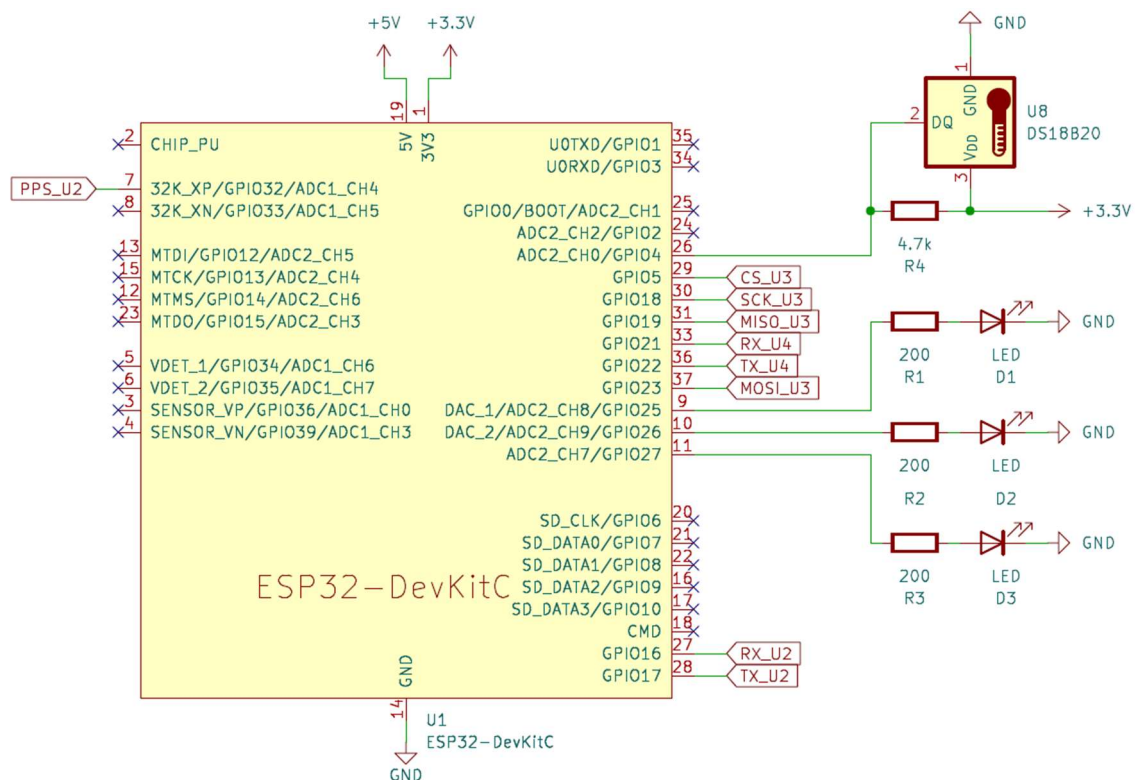
Zmiana wartości w tych polach powoduje wysłanie komunikatu aktualizacji danych do urządzenia. Oprogramowanie ESP32, dzięki zaimplementowanej funkcji `attributesCallback`, odbiera nowe nastawy (pod warunkiem aktywnego połączenia z siecią WiFi) i natychmiast dostosowuje cykl pracy koordynatora zadań, co pozwala na elastyczne zarządzanie zużyciem energii i częstotliwością próbkowania.

4.5. Budowa prototypu

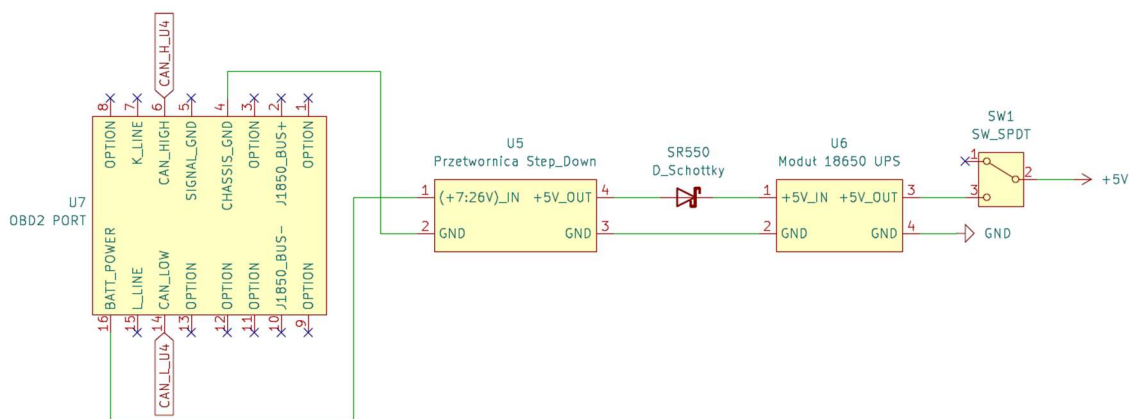
Proces realizacji urządzenia został podzielony na etap opracowania schematu połączeń elektrycznych (rysunki 4.8, 4.9, 4.10, 4.11) oraz fizyczny montaż prototypu. Prototyp wykonano z wykorzystaniem płytki uniwersalnej, co pozwoliło na trwałe połączenie komponentów przy zachowaniu możliwości modyfikacji połączeń. Najpierw zamontowano główne moduły, a następnie całość uzupełniono o diody sygnalizacyjne LED (dla statusu Wi-Fi, GPS i zapisu SD), diodę Schottky'ego oraz przełącznik zasilania.



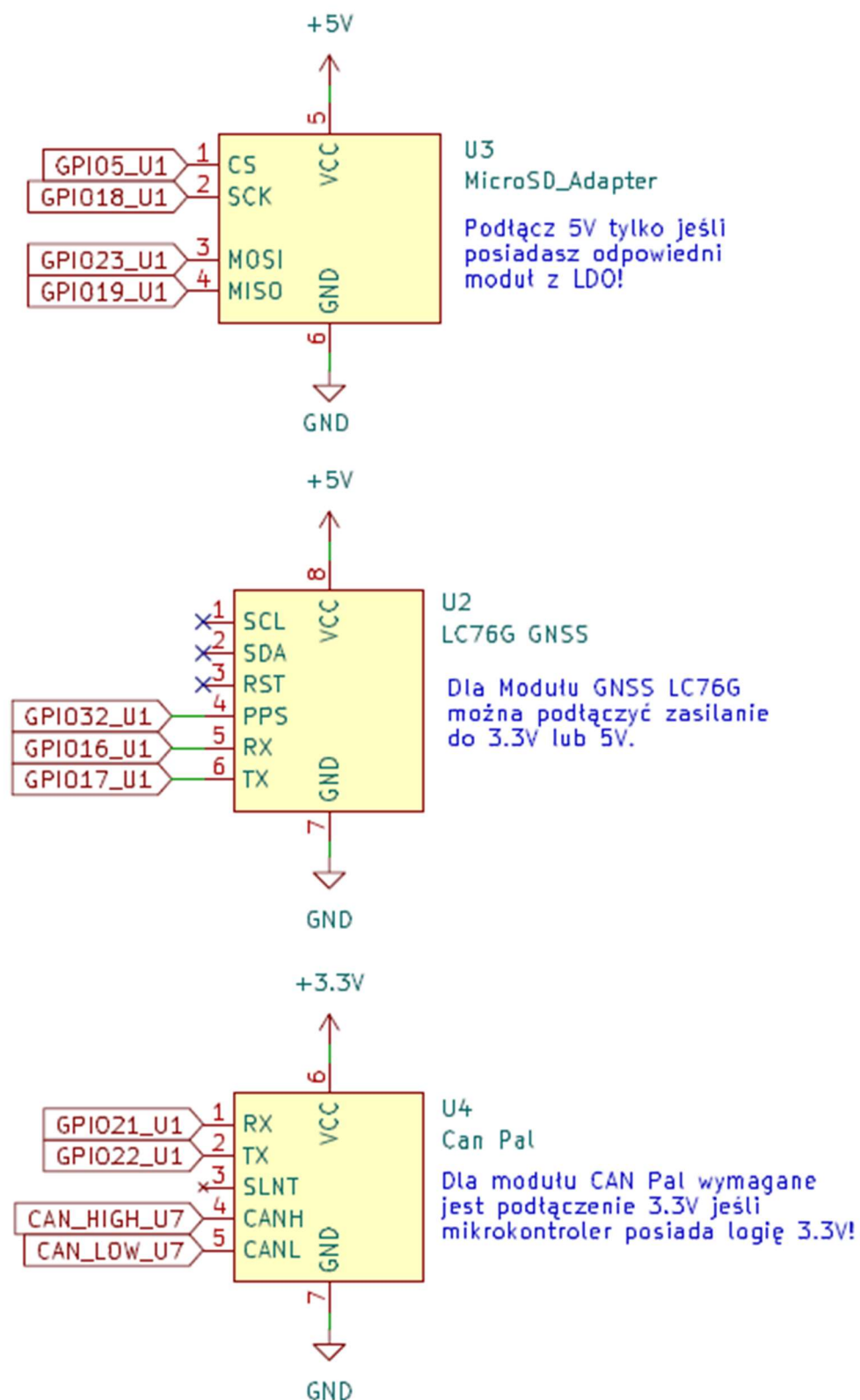
Rys. 4.8. Schemat połączeń elementów.



Rys. 4.9. Schemat połączeniowy ESP32.

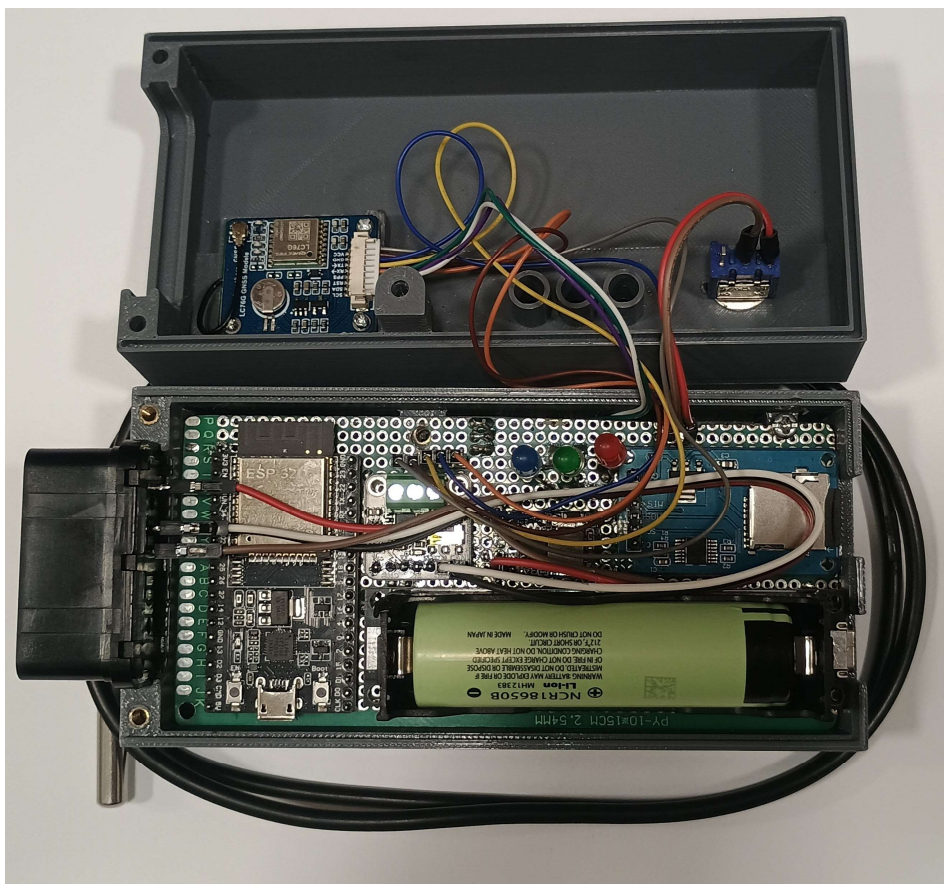


Rys. 4.10. Schemat połączeniowy zasilania.



Rys. 4.11. Schemat połączeniowy układów peryferyjnych.

Gotowy prototyp urządzenia przedstawiono na rysunkach 4.12 oraz 4.13.



Rys. 4.12. Widok otwartego prototypu urządzenia. Widoczne osadzone ESP32 oraz moduły peryferyjne na płytce uniwersalnej.



Rys. 4.13. Urządzenie przygotowane do testów.

4.6. Testy

W celu potwierdzenia zgodności wykonanego urządzenia z założeniami projektowymi, przeprowadzono testy. Proces weryfikacji podzielono na testy laboratoryjne, obejmujące sprawdzenie podsystemów elektrycznych, oraz testy drogowe, mające na celu ocenę pracy urządzenia w warunkach rzeczywistej eksploatacji w pojeździe.

4.6.1. Weryfikacja elektryczna i uruchomieniowa

Pierwszym etapem testów była kontrola poprawności montażu oraz stabilności zasilania. Przed instalacją mikrokontrolera w podstawce, zweryfikowano multimetrem napięcie wyjściowe fabrycznie skonfigurowanej przetwornicy Step-Down. Potwierdzono stabilną wartość 5 V, co jest napięciem bezpiecznym dla liniowego stabilizatora napięcia (ang. Low Dropout Regulator) znajdującego się na płycie rozwojowej ESP32.

Następnie zweryfikowano działanie układu podtrzymania zasilania (UPS – Uninterruptible Power Supply). Symulowano nagłe odłączenie zasilania głównego (samochodowego), potwierdzając, że urządzenie płynnie przełącza się na zasilanie z ogniwa Li-Ion 18650, nie powodując przy tym resetu mikrokontrolera.

W kolejnym kroku przeprowadzono pomiary poboru prądu w dwóch charakterystycznych scenariuszach pracy, co pozwoliło oszacować czas autonomii urządzenia na zasilaniu awaryjnym. Procedura pomiarowa obejmowała podłączenie sond multimetru do nóżek przełącznika (przy wyłączonym przełączniku). Urządzenie pomiarowe było ustawione na pomiar prądu stałego o wartości 10 A.

Poniżej przedstawiono metodologię wyznaczenia średniego poboru prądu oraz czasu pracy dla ogniwa Panasonic NCR18650B o pojemności nominalnej 3400 mAh.

1. **Scenariusz OFFLINE:** symuluje pracę poza zasięgiem zaufanej sieci Wi-Fi. W tym trybie urządzenie większość czasu spędza w stanie uśpienia GPS, wybudzając się cyklicznie na akwizycję danych i rzadkie skanowanie sieci. Obliczenia zostały wykonane dla okresu 300 s.

Zdarzenie A (Akwizycja danych, zapis na kartę SD oraz skanowanie sieci): 160 mA przez 2s co 5 minut.

- Liczba wystąpień w 300 s: 1
- Całkowity czas trwania: 5 [s]
- Ładunek: $5 * 160 = 800 [mAs]$

Zdarzenie B (Akwizycja danych oraz zapis na kartę SD): 80 mA przez 5 s co 30 s.

- Liczba wystąpień w 300 s: $\frac{300}{30} - 1 = 9$
- Całkowity czas trwania: $9 * 5 = 45 [s]$
- Ładunek: $45 * 80 = 3600 [mAs]$

Zdarzenie C (Akwizycja danych): 60 mA przez 4s co 15s.

- Liczba wystąpień w 300 s: $\frac{300}{15} - 9 - 1 = 10$
- Całkowity czas trwania: $10 * 4 = 40 [s]$
- Ładunek: $40 * 60 = 2400 [mAs]$

Zdarzenie D (Uśpienie GPS): 50 mA w pozostałym czasie.

- Całkowity czas trwania: $300 - 5 - 45 - 40 = 210 [s]$
- Ładunek: $210 * 50 = 10500 [mAs]$

Średni pobór prądu:

$$I_{avg} = \frac{800 + 3600 + 2400 + 10500}{300} = 57,67 [mA]. \quad (4.1)$$

Szacowany czas pracy:

$$t_{est} = \frac{3400}{57,67} = 58,96 [h]. \quad (4.2)$$

2. Scenariusz ONLINE: symuluje pracę w zasięgu sieci zaufanej. W tym trybie urządzenie cały czas jest podłączone do sieci WiFi co zwiększa prąd bazowy układu. Akwizycja danych przebiega cyklicznie co ustalony interwał.

Zdarzenie A (Akwizycja danych, zapis na kartę SD oraz transmisja): 100 mA przez 6 s co 30 s.

- Liczba wystąpień w 30 s: 1
- Czas trwania: 6 [s]
- Ładunek: $6 * 100 = 600 [mAs]$

Zdarzenie B (Akwizycja danych oraz zapis na kartę SD): 80 mA przez 4 s co 15 s.

- Liczba wystąpień w 30 s: 1
- Czas trwania: 4 [s]
- Ładunek: $4 * 80 = 320 [mAs]$

Zdarzenie C (Uśpienie GPS oraz WiFi w stanie spoczynku): 65 mA w pozostałym czasie.

- Całkowity czas trwania: $30 - 5 - 5 = 20 [s]$
- Ładunek: $20 * 65 = 1300 [mAs]$

Średni pobór prądu:

$$I_{avg} = \frac{600 + 320 + 1300}{30} = 74 [mA]. \quad (4.3)$$

Szacowany czas pracy:

$$t_{est} = \frac{3400}{74} = 45,95 [h]. \quad (4.4)$$

4.6.2. Testy funkcjonalne oprogramowania

Testy funkcjonalne polegały na weryfikacji działania poszczególnych modułów programowych przy użyciu wbudowanych mechanizmów diagnostycznych, takich jak logi na porcie szeregowym oraz diody statusowe LED:

Sygnalizacje stanu urządzenia zweryfikowano za pomocą diód:

- **LED_WiFi (GPIO 27):** Zapala się po poprawnym uzyskaniu adresu IP,

- **LED_GPS (GPIO 26):** Sygnalizuje uzyskanie poprawnego położenia geograficznego,
- **LED_SD (GPIO 25):** W stanie normalnym dioda ta świeci światłem ciągłym, sygnalizując poprawną inicjalizację karty i gotowość systemu plików. Zgaśnięcie diody informuje o błędzie krytycznym zapisu lub braku nośnika.

Symulacja awarii nośnika: przeprowadzono test polegający na fizycznym wyjęciu karty microSD podczas pracy urządzenia. System prawidłowo wykrył brak nośnika (zgaszenie diody LED_SD i ustawienie flagi błędu w telemetry). Po ponownym włożeniu karty, mechanizm „SdModule::ensureReady” automatycznie przywrócił sprawność systemu plików, wznowiając logowanie danych bez konieczności restartu zasilania.

Watchdog Timer: sprawdzono odporność systemu na zawieszenie. Wymuszona pętla nieskończona w jednym z zadań (Tasks) spowodowała poprawną reakcję układu Watchdog i restart systemu po upływie zdefiniowanego czasu (120 sekund).

Weryfikacja zarządzania zasobami: W celu potwierdzenia stabilności oprogramowania podczas pracy pod dużym obciążeniem, przeprowadzono monitorowanie zużycia pamięci operacyjnej RAM. Krytycznym scenariuszem testowym był moment synchronizacji danych z pliku buforowego („Pending”), który wymaga jednoczesnej obsługi systemu plików (odczyt z karty SD) oraz transmisji sieciowej (obsługa stosu TCP/IP).

Poniższy log z portu szeregowego przedstawia zrzut diagnostyczny z portu szeregowego zarejestrowany podczas trwania procesu synchronizacji danych z ThingsBoard:

```
----- MEMORY STATUS -----
[HEAP] Free: 109972 bytes, Min Free: 93592 bytes
[TASK] Coordinator: 2336 bytes free stack
[TASK] WiFi:      2184 bytes free stack
[TASK] DataSync:  3624 bytes free stack
[TASK] GPS:       2044 bytes free stack
[TASK] Temp:      2084 bytes free stack
[TASK] WebServer: 2572 bytes free stack
[TASK] Sleep:     1524 bytes free stack
-----
```

Analiza logów pozwala na sformułowanie następujących wniosków:

Brak wycieków pamięci: Wartość wolnej sterty ([HEAP] Free) utrzymuje się na poziomie ok. 109 kB, a najniższa zarejestrowana wartość (Min Free) wynosi ponad 93 kB, co stanowi bezpieczny margines dla układu ESP32.

Bezpieczeństwo wątków: Każde z zadań FreeRTOS posiada zapas wolnej pamięci stosu (free stack) wynoszący minimum 1500 bajtów. Oznacza to, że rozmiary stosów zadeklarowane w kodzie źródłowym zostały dobrane poprawnie i nie występuje ryzyko przepełnienia stosu (ang. Stack Overflow), co mogłoby prowadzić do niekontrolowanego restartu urządzenia.

Analiza zajętości pamięci masowej: Przeprowadzono analizę wielkości generowanych plików danych w formacie JSON Lines (.jsonl). Na podstawie pomiarów (poprzez opcje „Właściwości” w MS Windows) ustalono, że pojedynczy pakiet telemetryczny (zawierający znacznik czasu, współrzędne, prędkość, temperaturę i statusy) zajmuje średnio 208 bajtów.

Przy założeniu wykorzystania karty pamięci o pojemności użytkowej 29,1 GB oraz ciągłej pracy urządzenia z domyślnym interwałem zapisu wynoszącym 15 sekund, przeprowadzono szacunkowe obliczenia czasu zapełnienia nośnika:

Pojemność karty SD: 29,1 [GB] $\approx 3,12 * 10^{10}$ [B]

Częstotliwość zapisu: 1 na 15 [s]

Liczba pakietów na dobę: $4 * 60 * 24 * 365 = 2\,102\,400$

Rozmiar pojedynczego pakietu: 208 [B]

Maksymalna liczba pakietów:

$$N_{max} = \frac{3,12 * 10^{10}}{208} = 150\,000\,000. \quad (4.5)$$

Szacowany czas pracy do zapełnienia karty:

$$t_{est} = \frac{150\,000\,000}{2\,102\,400} \approx 71 \text{ lat}. \quad (4.6)$$

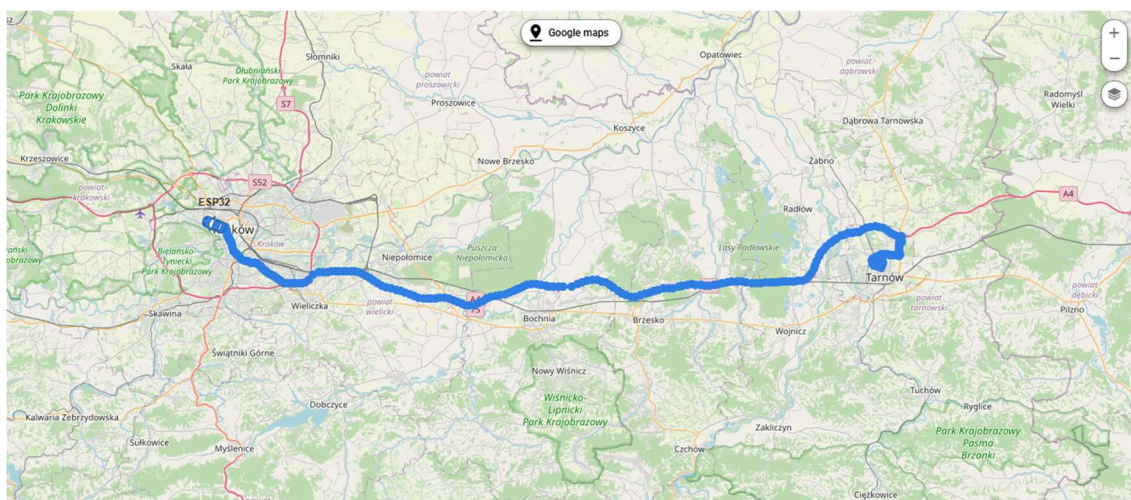
Powyższe obliczenia wskazują, że przy przyjętym formacie danych i interwale próbkowania, pojemność karty SD jest w praktyce niewyczerpywalna w całym cyklu życia urządzenia, nawet przy uwzględnieniu narzutu systemu plików (FAT32).

4.6.3. Testy w warunkach eksploatacyjnych

Kluczowym elementem weryfikacji był test drogowy mający na celu sprawdzenie mechanizmu buforowania danych. Przeprowadzono przejazd testowy na trasie o długości ok. 95 km (rysunek 4.14).

Procedura testowa:

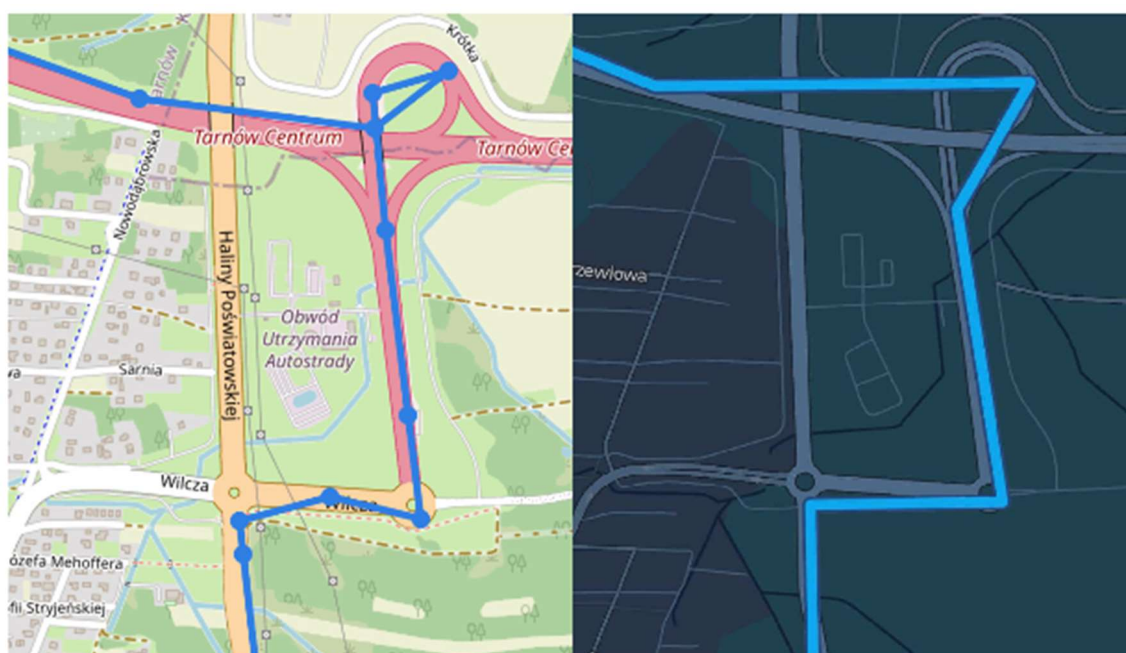
1. uruchomienie urządzenia w zasięgu sieci (Synchronizacja czasu, wysyłka próbna),
2. celowe wyłączenie punktu dostępowego Wi-Fi na czas jazdy,
3. weryfikacja zachowania systemu: urządzenie, nie mogąc połączyć się z brokerem MQTT, zaczęło zapisywać dane do pliku buforowego pending.jsonl na karcie SD,
4. ponowne włączenie sieci Wi-Fi,
5. po dotarciu do punktu docelowego urządzenie, automatycznie wykryło dostępność sieci i przesłało zaległe pakiety danych z pliku „pending” do platformy ThingsBoard, zachowując chronologię zdarzeń.



Rys. 4.14. Trasa zarejestrowana poprzez prototyp urządzenia i wyświetlona na platformie ThingsBoard.

4.5.4. Ocena jakości danych GNSS

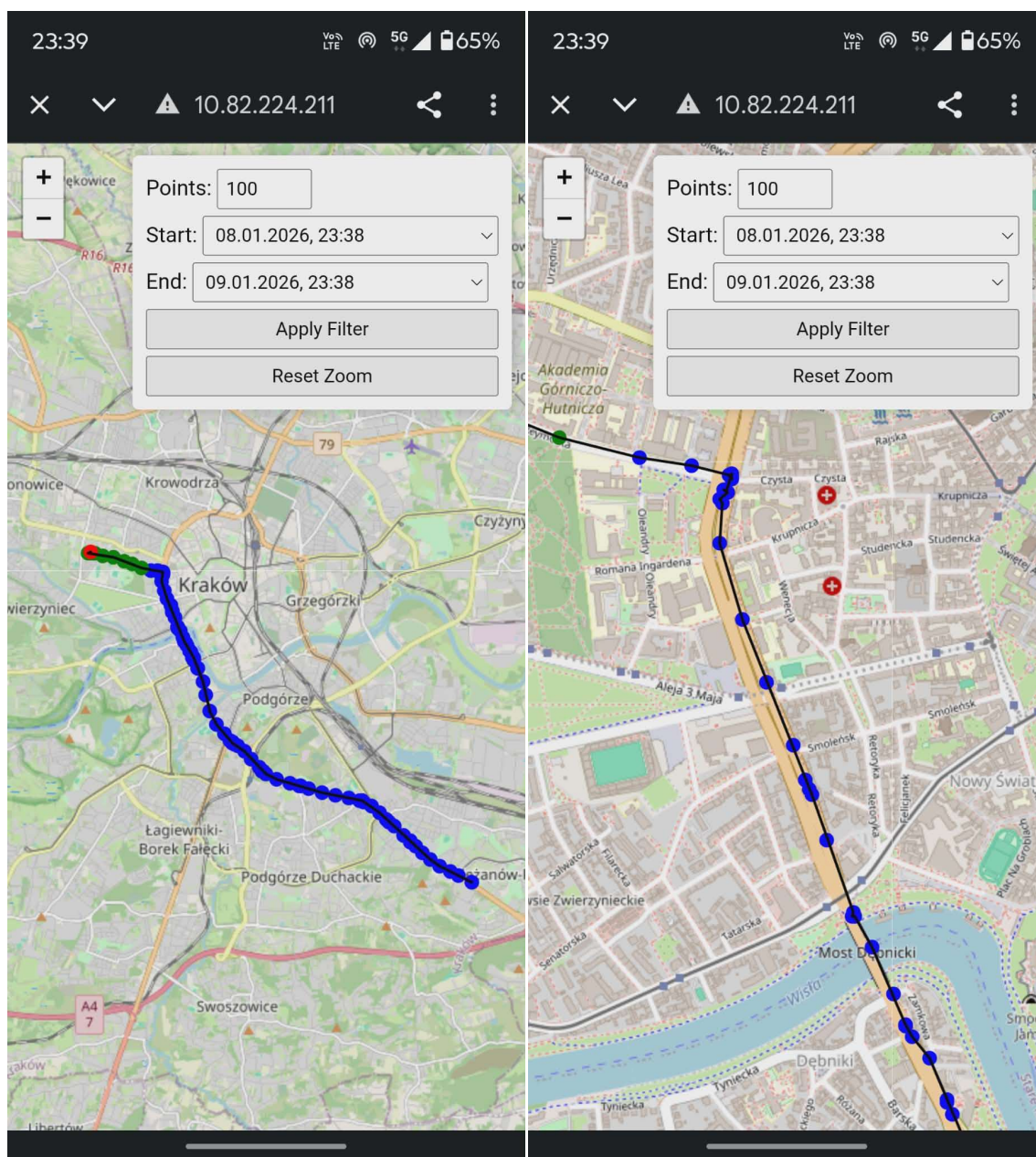
Dokładność modułu LC76G oceniono poprzez porównanie zarejestrowanej ścieżki na platformie ThingsBoard z trasą referencyjną pobraną z Google Maps Timeline (rysunek 4.15). Analiza wykazała, że w otwartym terenie błąd pozycjonowania w większości przypadków nie przekraczał 5 metrów. Natomiast zauważono również pojedyncze przypadki w których błąd pozycji wynosił nawet 30 metrów. Zauważono również poprawną pracę algorytmu „Cold Start” – czas uzyskania pierwszej pozycji po wybudzeniu urządzenia wynosił średnio poniżej 30 sekund, co jest wartością zgodną z dokumentacją modułu.



Rys. 4.15. Porównanie wycinka trasy zarejestrowanego w ThingsBoard (po lewej) oraz lokalizację w Google Maps (po prawej).

4.6.4. Wizualizacja i telemetria

Ostatnim etapem była weryfikacja poprawności danych prezentowanych na platformie ThingsBoard oraz na lokalnym serwerze WWW urządzenia (rysunek 4.16). Potwierdzono, że atrybuty takie jak m.in. prędkość, wysokość oraz temperatura są poprawnie dekodowane z formatu JSON i aktualizowane w czasie rzeczywistym (przy dostępnym połączeniu sieciowym).



Rys. 4.16. Widok mobilnego serwera HTTP hostowanego na ESP32.

5. Podsumowanie i wnioski

W ramach niniejszej pracy dyplomowej zaprojektowano, wykonano i przetestowano prototyp systemu lokalizacji i telemetrii pojazdu, oparty na mikrokontrolerze ESP32 oraz systemie operacyjnym czasu rzeczywistego FreeRTOS. Głównym celem projektu było zbudowanie rozwiązania modułowego, które łączyłoby funkcjonalność rejestratora parametrów jazdy z możliwością bezprzewodowej synchronizacji danych z chmurą IoT.

Zrealizowany prototyp spełnia wszystkie założenia funkcjonalne:

- **Akwizycja danych:** system poprawnie interpretuje strumień NMEA z modułu GNSS oraz odczytuje ramki z magistrali CAN, integrując te dane w spójne pakiety telemetryczne,
- **Niezawodność zapisu:** zaimplementowany mechanizm transmisji oraz zapisu skutecznie gromadzi dane na karcie pamięci w przypadku braku łączności, a następnie automatycznie synchronizuje je z serwerem ThingsBoard po powrocie do bazy,
- **Architektura oprogramowania:** wykorzystanie RTOS pozwoliło na równoległą obsługę zadań krytycznych czasowo (CAN, GPS) oraz zadań o wysokim czasie wykonania (zapis na SD, obsługa stosu TCP/IP).

5.1. Napotkane problemy i ograniczenia realizacyjne

W toku prac projektowych i wdrożeniowych zidentyfikowano szereg wyzwań technicznych oraz ograniczeń, które wpłynęły na ostateczny kształt prototypu. Poniżej omówiono kluczowe trudności napotkane podczas realizacji pracy.

1. Ograniczenia platformy serwerowej (ThingsBoard Demo)

Istotnym wyzwaniem okazały się limity narzucone przez wersję demonstracyjną (Community Edition) platformy ThingsBoard. Zidentyfikowano ograniczenie przepustowości przyjmowania danych telemetrycznych, które uniemożliwiło przesyłanie dużych paczek danych w jednym komunikacie MQTT.

- **Problem:** Próba wysłania tablicy JSON zawierającej więcej niż 2 pakiety pomiarowe skutkowała odrzuceniem połączenia przez brokera lub utratą pakietów.
- **Rozwiązanie:** Wymusiło to programowe ograniczenie parametru SEND_BATCH_SIZE w oprogramowaniu układowym. System został skonfigurowany tak, aby dzielić dane historyczne (zbuforowane na karcie SD) na mniejsze fragmenty, co wydłużyło czas synchronizacji danych po powrocie urządzenia do trybu online, ale zapewniło stabilność transmisji.

2. Kompatybilność z nowoczesnymi systemami bezpieczeństwa pojazdów (Secure Gateway)

Podczas prób integracji z nowszymi modelami pojazdów (wyprodukowanymi po 2017 roku) napotkano barierę w postaci modułu SGW (Security Gateway).

- **Problem:** Nowoczesne architektury samochodowe blokują nieautoryzowany dostęp do magistrali CAN, uniemożliwiając odczyt parametrów oraz inżynierię wsteczną ramek bez posiadania certyfikatów producenta i dedykowanego sprzętu odblokowującego.
- **Wniosek:** Ostatecznie testy funkcjonalne ograniczono do pojazdów starszej generacji, co wskazuje na konieczność stosowania w przyszłości certyfikowanych bramek OBD2 typu „passthru” dla nowszych flot lub bezpośredniego podpięcia się do linii CAN przy np. silniku.

3. Wyzwania związane z miniaturyzacją (PCB)

Jednym z założeń projektowych było dążenie do miniaturyzacji urządzenia. Mimo opracowania schematu połączeń, w ramach niniejszej pracy nie udało się zrealizować dedykowanego obwodu drukowanego (PCB – Printed Circuit Board).

- **Przyczyna:** Ograniczenia czasowe i budżetowe wymusiły pozostanie przy konstrukcji prototypowej opartej na płycie uniwersalnej.
- **Skutek:** Urządzenie posiada większe gabaryty niż komercyjne odpowiedniki.

5.2. Wnioski z realizacji

Analiza działania prototypu w warunkach laboratoryjnych i polowych pozwala na sformułowanie następujących wniosków.

- **Wydajność platformy sprzętowej:** układ ESP32 dysponuje wystarczającą mocą obliczeniową do przetwarzania danych w czasie rzeczywistym, połączeń MQTT oraz obsługi systemu plików, pozostawiając margines zasobów na przyszłą rozbudowę.
- **Model komunikacji Wi-Fi:** decyzja o wykorzystaniu Wi-Fi jako głównego medium transmisyjnego obniżyła koszty eksploatacji (brak abonamentu GSM) i uprościła konstrukcję urządzenia. Ogranicza to jednak zastosowanie systemu do scenariuszy typu „czarna skrzynka”, wykluczając śledzenie antykradzieżowe w czasie rzeczywistym.
- **Złożoność integracji CAN/OBD:** choć fizyczny odczyt ramek CAN działa poprawnie, pełna integracja ze standardem OBD2 (ISO 15765) wymaga złożonej implementacji oraz dostosowania zapytań PID do specyfiki konkretnego producenta pojazdu.

5.3. Kierunki dalszego rozwoju

Otwarta architektura projektu oraz modułowa budowa kodu źródłowego stwarzają szerokie możliwości rozwoju. Poniżej przedstawiono kluczowe obszary, których implementacja pozwoliłaby na przekształcenie prototypu w pełni funkcjonalny produkt rynkowy.

- **Moduł LTE-M / NB-IoT:** zastąpienie lub uzupełnienie Wi-Fi modułem komórkowym zapewniłoby ciągłą transmisję danych niezależnie od lokalizacji. Umożliwiłoby to realizację funkcji antykradzieżowych, takich jak powiadomienia „Push” o nieautoryzowanym ruchu pojazdu czy wyjeździe poza wyznaczoną geostrefę (ang. Geofencing).

- **Aktualizacje OTA (Over The Air):** implementacja mechanizmu zdalnej aktualizacji oprogramowania układowego przez Wi-Fi pozwoliłaby na naprawę błędów i dodawanie nowych funkcji bez konieczności fizycznego dostępu do urządzenia.
- **Implementacja algorytmu cyklicznego uśpienia:** docelowe rozwiązanie powinno opierać się na automatycznej detekcji postoju. Proponuje się wdrożenie logiki, w której system po wykryciu braku przemieszczania się (na podstawie danych GNSS) przez okres 5 minut automatycznie przechodzi w stan głębokiego uśpienia (ang. Deep Sleep). W celu zachowania łączności, układ będzie wybudzany cyklicznie co 10 minut dla wysłania pakietu statusowego, po czym ponownie przejdzie w stan uśpienia.
- **Sprzętowy monitoring stanu baterii:** w celu zwiększenia niezawodności pracy na zasilaniu akumulatorowym, układ należy rozbudować o dzielnik napięcia podłączony do przetwornika ADC (Analog to Digital Converter) mikrokontrolera ESP32. Umożliwi to ciągły pomiar napięcia ogniwa Li-Ion. Pozwoli to na bezpieczne zamknięcie systemu plików i zakończenie pracy urządzenia przed spadkiem napięcia poniżej poziomu krytycznego, co chroni akumulator przed głębokim rozładowaniem, a dane na karcie SD przed uszkodzeniem.
- **Szyfrowanie pamięci masowej:** w obecnej wersji pliki na karcie SD są zapisywane jawnym tekstem (JSON). Wdrożenie szyfrowania symetrycznego (np. AES-128 – Advanced Encryption Standard) zabezpieczyłoby dane o trasach i parametrach pojazdu w przypadku kradzieży nośnika.
- **Bezpieczna transmisja:** implementacja protokołu MQTTS (MQTT over SSL/TLS) z weryfikacją certyfikatów serwera zabezpieczyłaby transmisję przed atakami podczas wysyłki.
- **Dedykowane PCB:** przeniesienie układu z płytki uniwersalnej na zaprojektowany obwód drukowany (PCB) z wykorzystaniem technologii montażu powierzchniowego (SMD – Surface Mounted Device) pozwoliłoby na znaczną miniaturyzację urządzenia i zwiększenie jego odporności na wibracje występujące w pojeździe.

6. Wykaz Publikacji

- [1] Ciemcioch K. Internet Rzeczy – zastosowanie, zagrożenia, bezpieczeństwo, Zbliżenia Cywilizacyjne, 2023, 19(3), 61–85.
- [2] Nores: Telemetria w IoT – co to jest i jak napędza Przemysł 4.0?, Gorzów Śląski: Nores Sp. z o.o. Dostępny: <https://nores.pl/telemetria-w-iot-co-to-jest-i-jak-napedza-przemysl-4-0/> (odwiedzona 11.01.2026).
- [3] National Instruments: Controller Area Network (CAN) Protocol Overview. Austin: National Instruments Corp. Dostępny: <https://www.ni.com/en/shop/seamlessly-connect-to-third-party-devices-and-supervisory-system/controller-area-network--can--overview.html> (odwiedzona 11.01.2026).
- [4] Sital Technology: CAN Bus Voltage Level: Analyzing the Electrical Potential. Dostępny: <https://sitaltech.com/can-bus-voltage-level-analyzing-the-electrical-potential/> (odwiedzona 11.01.2026).
- [5] Mendel M.: Magistrala CAN (Controller Area Network) konfiguracja i transmisja danych. Zielonka: Wojskowy Instytut Techniczny Uzbrojenia. Dostępny: <https://bibliotekanauki.pl/articles/235634.pdf> (odwiedzona 11.01.2026).
- [6] Usmani M.F.: MQTT Protocol for the IoT - Review Paper. ResearchGate. Dostępny: https://www.researchgate.net/publication/373640610_MQTT_Protocol_for_the_IoT_Review_Paper (odwiedzona 11.01.2026).
- [7] Space Economy Academy: Differences Between GNSS Systems. Darmstadt: Space Economy Academy. Dostępny: <https://seac-space.com/differences-between-gnss-systems/> (odwiedzona 11.01.2026).
- [8] Bernstein D.: NMEA Sentences. Harrisonburg: James Madison University. Dostępny: <https://w3.cs.jmu.edu/bernstdh/web/common/help/nmea-sentences.php> (odwiedzona 11.01.2026).

- [9] Quectel Wireless Solutions: LC26G&LC26G-T&LC76G&LC86G Series AGNSS Application Note. Shanghai: Quectel Wireless Solutions. Dostępny: https://mm.digikey.com/Volume0/opasdata/d220001/medias/docus/6504/LC26G-LC26G-T-LC76G-LC86G_Series_AGNSS_Application_Note_V1.1.pdf (odwiedzona 11.01.2026).
- [10] Macekova L., 1-Wire - The Technology for Sensor Networks, Acta Electrotechnica et Informatica, 2012, 12 (4), 52-55.
- [11] Barry R.: Mastering the FreeRTOS™ Real Time Kernel: A Hands-On Tutorial Guide. Bristol, Real Time Engineers Ltd 2016.
- [12] BearPoint: Lokalizator GPS BearPoint. Wałbrzych: BearPoint. Dostępny: <https://bearpoint.pl/produkt/lokalizator-gps-bearpoint/> (odwiedzona 11.01.2026).
- [13] Szpiegujemy.com: Lokalizator GPS GL300 Server z długim czasem czuwania. Wrocław: Nexus. Dostępny: <https://szpiegujemy.com/product-pol-1690> (odwiedzona 11.01.2026).
- [14] Szpiegujemy.com: Lokalizator GPS z hermetyczną obudową i magnesami pod samochód 20000mAh. Wrocław: Nexus. Dostępny: <https://szpiegujemy.com/product-pol-1696> (odwiedzona 11.01.2026).
- [15] Espressif Systems, ESP32-DevKitC V4 User Guide, Espressif Documentation. Dostępny: https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32/esp32-devkitc/user_guide.html. (odwiedzona 11.01.2026).
- [16] Waveshare, LC76G GNSS Module Wiki, Waveshare Wiki. Dostępny: https://www.waveshare.com/wiki/LC76G_GNSS_Module. (odwiedzona 11.01.2026).
- [17] Adafruit Industries, Adafruit CAN PAL Overview, Adafruit Learning System. Dostępny: <https://learn.adafruit.com/adafruit-can-pal/overview>. (odwiedzona 11.01.2026).

- [18] Analog Devices, DS18B20: Programmable Resolution 1-Wire Digital Thermometer Data Sheet, Analog Devices Technical Documentation. Dostępny: <https://www.analog.com/media/en/technical-documentation/data-sheets/ds18b20.pdf>. (odwiedzona 11.01.2026).
- [19] Benoît B.: ArduinoJson: Efficient JSON serialization for embedded C++. Dostępny: <https://arduinojson.org/v7/> (odwiedzona 11.01.2026).
- [20] O'Leary N.: PubSubClient: A client library for MQTT messaging. Biblioteka programistyczna. Dostępny: <https://github.com/knolleary/pubsubclient> (odwiedzona 11.01.2026).
- [21] Hart M.: TinyGPSPlus Library. Dostępny: <https://github.com/mikalhart/TinyGPSPlus> (odwiedzona 11.01.2026).
- [22] Stoffregen P. et al.: OneWire Library. Dostępny: <https://github.com/PaulStoffregen/OneWire> (odwiedzona 11.01.2026).
- [23] Burton M. et al.: DallasTemperature Library. Dostępny: <https://github.com/milesburton/Arduino-Temperature-Control-Library> (odwiedzona 11.01.2026).
- [24] Espressif Systems: Two-Wire Automotive Interface (TWAI). Shanghai: Espressif Systems (Shanghai) Co., Ltd. Dostępny: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/twai.html> (odwiedzona 11.01.2026).
- [25] ThingsBoard: ThingsBoard Documentation & Guides. Dostępny: <https://thingsboard.io/docs/guides/> (odwiedzona 11.01.2026).